

On new variants of funcCPA security and related CCA-secure constructions*

Caroline Fontaine¹, Marc Renard^{1,2}, Renaud Sirdey³ and Oana Stan³

¹ Université Paris-Saclay, CNRS, ENS Paris-Saclay, Laboratoire Méthodes Formelles, 91190, Gif-sur-Yvette, France

`caroline.fontaine@cnrs.fr, marc.renard@lmf.cnrs.fr`

² Université de Rennes, French Institute for Research in Computer Science and Automation, Institut de Recherche en Informatique et Systèmes Aléatoires,

³ Université Paris-Saclay, CEA, List, 91477, Palaiseau, France,

`renaud.sirdey@cea.fr, oana.stan@cea.fr`

Abstract. FuncCPA is a recent security notion in which the CPA game is extended by a functional re-encryption oracle in order to model setups in which a server performing FHE computations is allowed to interactively delegate part of the computation back to the client. In this paper, we study funcCPA-style variants of several CCA security notions, including CCA1 and the more recent vCCA security. Contrary to the CPA case where a strict separation holds between CPA and funcCPA, we show that these new variants are equivalent to their respective originating CCA security notions. Interestingly, funcCPA-style security notions also model setups where, rather than delegating part of the encrypted domain computation all the way back to the client, the server has the ability to perform this delegation towards an honest or semi-honest client proxy it hosts, such as a secure enclave. We then provide a number of blueprints for achieving both FHE-like capabilities and advanced CCA security properties which may then meaningfully be implemented by leveraging on the combination of a partially homomorphic scheme and a semi-honest non-colluding enclave hosted within the server performing the encrypted domain calculations itself.

1 Introduction

The study of “client-aided” homomorphic encryption from a security notion point of view has been ignited in [1] which introduced the notion of funcCPA security.

* This work was supported by the France 2030 ANR Project ANR-22-PECY-003 SecureCompute and funded partially by the French government through the France 2030 program, under the project RESQUE, and by the European Union under Grant Agreement No. 101167964. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Cybersecurity Competence Centre. Neither the European Union nor the European Cybersecurity Competence Centre can be held responsible for them. This work was done in part while the second author was at CEA List and ENS Paris-Saclay.

In this notion, a CPA adversary is further granted access to a *decrypt oracle* which refreshes ciphertexts, and optionally executes some function in the process, with the knowledge of the secret decryption key. This security notion is meaningful in several settings where a server performing homomorphic computations may, from time to time, be allowed to outsource certain tasks back to the client or to a local trusted-to-some-extent client proxy. As an example, a practically meaningful setting which has been considered in several theoretical [35] and practical works [31,60,34] is when a local client proxy such as a secure enclave is entrusted with the secret key of an homomorphic scheme and performs decryption operations to replace invocations of the more costly bootstrapping procedure. It is furthermore folklore knowledge that this decryption operation may be performed after some (one-time) masking is done by the server. More precisely, to re-encrypt a ciphertext c of a message $m \in \mathbb{Z}_t$ under a scheme $\mathcal{E}$ with plaintext domain, e.g., $\mathbb{Z}_t$, the server first picks a random μ in $\mathbb{Z}_t$ and sends the result of the homomorphic add-plaintext operation $c' = \mathcal{E}.\text{AddPlain}(c, \mu)$ to the local client proxy. The proxy then returns $c'' = \mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c'))$, i.e. a re-encryption of the decryption of that latter ciphertext, and only sees $m + \mu \bmod t$ in the process. Following this, the server computes on its own ciphertext $\mathcal{E}.\text{AddPlain}(c'', -\mu)$ to get a new encryption of the same message¹. This technique is then appropriate when the server hosts a non-colluding semi-honest local client proxy, a model which can be quite faithfully implemented by means of a secure enclave. The same kind of techniques, with masking included, has also been considered in the folklore [55] as a mean for relinearizing post-multiplication ciphertexts for (multiplicative) depth-1 schemes such as BGN [16], to allow further multiplications.

With respect to CCA security, it should be emphasized that all the FHE usable in practice² achieve only CPA security and are also known to be trivially CCA1 insecure. Although it is well-known that malleability is contradictory with CCA2 security, building *practically efficient* FHE constructions achieving some degree of CCA security (e.g. CCA1 or even above) remains a very important open question [62]. It may be interesting to highlight that the large majority of works investigating FHE constructions achieving some degree of CCA security [18,24,49,50,22] require either the random oracle model or some non falsifiable assumption due to their reliance on general zk -SNARKs [42] or even stronger primitives such as iO. This is to the exception of the recent (Crypto'25) paper of Yang et. al [62] which achieves the *tour de force* of giving a construction that is IV-CCA secure (a stronger-than-CCA1 notion they define and study) in the standard model, based solely on the LWE assumption. On the downside, their construction is intrinsically *non-compact*. The compactness requirement which is naturally expected from FHE, is however achieved by some of the construction blueprints that are proposed in [50,22] (namely Encrypt-then-Sign in [50] and Encrypt-then-Prove in [22]). However, these generic constructions are not easily amenable to efficient implementations in their full generality, essentially because

¹ The extent to which this is true depends on whether or not $\mathcal{E}$ satisfies some correctness property.

² Essentially BFV [19,37], BGV [20], CKKS [29] and TFHE [30].

they intimately require powerful SNARK machinery. Additionally, in light of recent results [47,26], it appears that CCA security properties may be more easily achieved (with lightweight machinery) in the realm of number-theoretic linear homomorphic schemes than in that of lattice-based (linear or) fully homomorphic ones.

In light of this context, a natural question to ask is whether we can leverage on folklore client/enclave-aided techniques such as the ones above to *provably* achieve meaningful CCA security properties and at the same time provide FHE-like capabilities?

In this paper, we provide positive answers to this question through the investigation of “funcCPA-style” variants of CCA security notions, including CCA1 and the more recent vCCA security notion of Manulis and Nguyen [50]. In both cases, we consider extensions where the adversary is granted access to either a decrypt or relinearization (hence univariate) oracle or a bivariate oracle which performs an operation after decryption of its two input ciphertexts, e.g. a multiplication, and re-encrypts the result. Contrary to the CPA case where there is a strict separation between CPA and funcCPA security, we show that these new CCA security variants are equivalent to their respective originating CCA security notions. We then turn to concrete client-aided “fully” homomorphic constructions examples based on the Paillier scheme [54] and its (multiplicative) depth-1 Catalano-Fiore variant [25], respectively associated with a multiplication or a relinearization oracle. We do so under several assumptions. First, we propose CCA1 secure constructions under the *heuristic* assumption that the Paillier scheme is CCA1 secure (an assumption that we briefly discuss the plausibility in light of recent developments). Note that this first round of constructions rely on Paillier in a black-box fashion, so we also discuss the extent to which other schemes can be used to instantiate them. Then, we investigate constructions built from a variant of the Paillier-ElGamal scheme that has recently been proven CCA1 secure under standard assumptions [47]. Finally, we provide vCCA secure constructions under the assumption that the Paillier scheme is CPA secure and that a *previously known* “two-ciphertexts” construct has the Linear-only Homomorphism property, an assumption that has been used for more than a decade at the core of several proof-of-knowledge constructions. In these latter constructions, the Paillier scheme may further easily be replaced by other CPA secure linear-only homomorphic schemes.

Overall, our results reveal that “old school” linearly homomorphic encryption schemes may be meaningfully combined with semi-honest client proxies (as implemented faithfully by means of secure enclaves) in simple and pragmatic constructions in order to both achieve advanced CCA security properties and emulate FHE capabilities.

1.1 Summary of contributions

The contributions of this paper are as follows:

- We introduce two variants of CCA1 security, namely CCA1^R (“CCA1 with a decrypt oracle”) and CCA1^M (“CCA1 with a multiplicative oracle” or

an oracle performing any other bivariate operation), which are suitable to model client- or enclave-aided homomorphic computations. We then show that these two new security notions remain equivalent to CCA1. Under the heuristic assumption that the Paillier scheme is CCA1 secure, and by associating a multiplication oracle with it, we achieve CCA1 secure client-aided “fully” homomorphic encryption. We then further propose several ways to include masking in the implementation of these oracles when instantiated in semi-honest local client proxies. We also discuss the extent to which the recent Libert’s variant [47], which has recently been proved CCA1 under more standard assumptions, can be plugged in such a construction.

- We also consider the Catalano-Fiore extension of Paillier which allows one-level of multiplication. We show that this construction achieves CCA1 security under the assumption that Paillier does. In this context, we further introduce the notion of CCA1^L (“CCA1 with a relinearization oracle”) security and also show that CCA1^L collapses onto CCA1. It thus provides another way to achieve CCA1 secure client-aided “fully” homomorphic encryption, if one wishes to rely on a relinearization rather than a multiplication oracle.
- Then, we focus on the “beyond CCA1” security notion of Manulis and Nguyen [50] and also introduce new variants, namely vCCA^R and vCCA^M , which we show equivalent to the vCCA base notion. We then consider associating a multiplication oracle with a “two-ciphertexts” variant of Paillier, denoted $\mathcal{P}^{(2)}$, which has recently been proven vCCA secure [26] under the assumption that Paillier is CPA secure and another previously known Linear-only Homomorphism assumption. This allows to achieve vCCA secure client-aided “fully” homomorphic encryption. Lastly, we show that this approach is also amenable to masking.
- As a last contribution, we also bring the Catalano-Fiore multiplicative extension of Paillier to the vCCA level under the assumption that $\mathcal{P}^{(2)}$ also achieves this level of security. To the best of our knowledge, this is the first concrete scheme supporting both homomorphic additions and multiplications (even limited to one-level) that is proven vCCA secure. Then, we define the notion of vCCA^L security and show that it collapses onto vCCA. Lastly, we adapt the associated relinearization oracle to allow masking.

For convenience, we provide in Fig. 1 a summary of the relationships among the security notions considered in this paper.

1.2 Paper organization

This paper is organized as follows. Sect. 2 covers preliminaries and related works on relevant security notions. Then, in Sect. 3, we introduce our new variants of CCA1 security and study their relations with this baseline notion. Sect. 4 discusses our client-aided constructs built on the Paillier scheme or its Catalano-Fiore multiplicative variant. Then, in Sect. 5, we study the extent to which the latter constructs can be ported to the Paillier-ElGamal variant of Libert. In Sect. 6 we introduce new variants of vCCA security, study their relations to

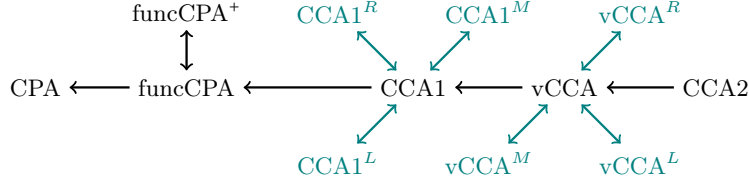


Fig. 1. Relations among the security notions considered in this paper. All new notions and relations are depicted in **green**.

baseline vCCA, and finally introduce client-aided constructions achieving this level of security. Sect. 7 concludes the paper before some appendix which provide additional material included for self-containedness.

2 Preliminaries and related works

Given $l, u \in \mathbb{Z}$, we use $\llbracket l, u \rrbracket$ to denote the set $\{l, l+1, \dots, u-1, u\}$. Reduction modulo q is denoted as $[\cdot]_q$. We use this notation explicitly only when it avoids possible ambiguities.

2.1 Basic definitions

We define an encryption scheme $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ over key space $\mathcal{K}$, plain-text domain $\mathcal{M}$ and ciphertext domain $\mathcal{C}$ as a triplet of PPT algorithms:

- **KeyGen**: on input 1^λ , outputs an encryption key ek and a decryption key sk .
- **Enc**: on inputs $m \in \mathcal{M}$ and ek , outputs an encryption $c \in \mathcal{C}$ of m .
- **Dec**: on inputs $c \in \mathcal{C}$ and sk , outputs a³ decryption $m \in \mathcal{M} \cup \{\perp\}$ of c .

Let COIN denote the randomness space of $\mathcal{E}$. We sometimes externalize the randomness used in the encryption function by means of the notation $\text{Enc}(m; r)$, with $m \in \mathcal{M}$ and $r \in \text{COIN}$. In this latter case, the function $\text{Enc} : \mathcal{M} \times \text{COIN} \rightarrow \mathcal{C}$ is deterministic⁴. When ek is public, we say that $\mathcal{E}$ is a *public-key* encryption scheme and use pk to denote ek . When, for all $(\text{ek}, \text{sk}) \in \mathcal{K}$, all $m \in \mathcal{M}$ and $r \xleftarrow{\$} \text{COIN}$, we have that,

$$\Pr(\text{Dec}(\text{Enc}(m; r)) \neq m) \leq \text{neg}(\lambda), \quad (1)$$

we say that $\mathcal{E}$ is *statistically correct* or simply *correct*. When the above probability is zero, we talk of *perfect* correctness.

Given a function class $\mathcal{F}_H$, we define a homomorphic encryption (HE) scheme $\mathcal{E}_H$ as an encryption scheme augmented by a *deterministic*⁵ polynomial-time

³ Decryption may not be deterministic.

⁴ In this case we assume that r is uniformly picked in COIN to provide a seed that the encryption function then uses to sample from the proper distributions.

⁵ As is the case for the mainstream linear or fully homomorphic schemes.

algorithm **Eval** which, on input $f \in \mathcal{F}_H$ and $c_1, \dots, c_L \in \mathcal{C}^L$, where L denotes the arity of function f , outputs a new *evaluated* ciphertext. When $\mathcal{E}_H$ satisfies condition (1) and when **Eval** is such that for all $(\text{ek}, \text{sk}) \in \mathcal{K}$, all $f \in \mathcal{F}_H$, all $(m_1, \dots, m_L) \in \mathcal{M}^L$ and $(r_1, \dots, r_L) \xleftarrow{\$} \text{COIN}^L$,

$$\Pr(\text{Dec}(\text{Eval}(f, \text{Enc}(m_1; r_1), \dots, \text{Enc}(m_L; r_L))) \neq f(m_1, \dots, m_L)) \leq \text{neg}(\lambda), \quad (2)$$

we say that $\mathcal{E}_H$ is a *correct* HE scheme. When this is not the case, we say that $\mathcal{E}_H$ is an *approximate* HE scheme. Consistently with [46], to avoid arbitrary schemes with unreliable **Eval** to be marketed as approximate HE schemes, we add an additional condition that, for some (small) $\varepsilon \geq 0$, the following holds,

$$\Pr(\|\text{Dec}(\text{Eval}(f, \text{Enc}(m_1; r_1), \dots, \text{Enc}(m_L; r_L))) - f(m_1, \dots, m_L)\|_\infty \leq \varepsilon) \geq \mu, \quad (3)$$

with⁶ $\mu \geq \frac{3}{4}$. Lastly, a scheme such that $\varepsilon = 0$ and $\frac{3}{4} \leq \mu < 1 - \text{neg}(\lambda)$ is said to be *somewhat correct*.

Although the security notions we study in this paper make sense in the general setting where neither perfect nor statistical correctness hold, in our concrete constructions, unless otherwise stated, we will consider perfectly correct HE schemes.

2.2 Security notions

Unless otherwise stated, we consider only (single-challenge) indistinguishability-based security notions and thus omit the IND prefix. In all the security games we consider, the adversary has to guess a random bit γ , privately chosen by its challenger, with non-negligible advantage, given some oracles including at least a challenge oracle which given two distinct messages m_0 and m_1 returns an encryption c^* of m_γ .

For completeness, we quickly recall the standard notions of CPA and CCA1 security in Appendix E. In that latter section, we also briefly touch on the notion of CPA^D security which is important for FHE [45].

vCCA, vCCA^D (and IV-CCA) security. As introduced in [50], vCCA security is a single challenge security notion. As such, the vCCA game has two decryption oracles. The second step (post-challenge publication) oracle assumes the existence of a PPT witness extractor $\text{Extract} : \mathcal{C} \times \mathcal{X} \rightarrow \mathcal{F}_H \times \mathcal{C}^*$, where $\mathcal{X}$ denotes a set of auxiliary data⁷ such that:

⁶ In practice μ is typically chosen above $1 - 2^{-40}$. In some contexts, e.g. [3], μ even has to be at least $1 - \text{neg}(\lambda)$.

⁷ The spirit of the vCCA security notion is (at least) to model construction blueprints which embed proof material in their ciphertexts and rely on a SNARK to enforce correct homomorphic evaluations over some input ciphertexts. In this context, the above **Extract** thus corresponds to the extractor of that underlying SNARK which allows to retrieve a witness from the proven statement as well as auxiliary data forming the trace of the execution of the adversary, see [50, 22] for more details.

- For any ciphertext $c \in \mathcal{C}$ which is obtained by invoking $\text{Eval}(f, c_0, \dots, c_{L-1})$,

$$\text{Extract}(c, \text{aux}) = (f, c_0, \dots, c_{L-1}).$$

- Otherwise, $\text{Extract}(c, \text{aux}) = (\text{id}, c)$.

Before the unique challenge request, the first step decryption oracle unconditionally returns $\mathcal{E}.\text{Dec}(c)$ for any ciphertext c submitted by the adversary while, after generation of the unique challenge ciphertext c^* , the vCCA second step oracle is defined as,

$$\text{Dec}_{\text{vCCA}}^{\mathcal{O}}(c^*, c) = \begin{cases} \mathcal{E}.\text{Dec}(c) & \text{if } c^* \notin \{c_0, \dots, c_{L-1}\}, \\ \perp & \text{otherwise,} \end{cases} \quad (4)$$

with $(f, c_0, \dots, c_{L-1}) \leftarrow \text{Extract}(c, \text{aux})$. In essence, the vCCA game is exactly the single challenge CCA2 game, with the second step decryption oracle being augmented in order to filter out *all* byproducts of the challenge ciphertext (rather than just the challenge ciphertext). In [50], vCCA security is defined and studied under the correctness assumption of the underlying FHE scheme, and then further studied in [22] when that assumption is not satisfied. Then, [22] also defines the notion of vCCA^D security, which, in a nutshell, is a “CPA^D-style” multiple challenge variant of vCCA in which the decryption oracle also accepts byproducts of the challenge ciphertexts as long as the associated left and right cleartext evaluations coincide (i.e. that the expected cleartext output does not leak the challenge bit).

Taken together, these two notions are the strongest CCA security notions so far achievable, respectively, by correct and approximate homomorphic schemes. However, the only known construction strategies intimately require advanced SNARK machinery, undermining their practicality. Following [22], vCCA and vCCA^D security are equivalent when the correctness assumption holds, so we only consider the former security notion in this paper.

To conclude this section, it is important to also mention the notion of IV-CCA (CCA1<IV-CCA<vCCA) security introduced in the recent (Crypto’25) paper of Yang et. al [62]. However, the spirit of this notion is to model *non-compact* FHE. Indeed, in IV-CCA, the second step decryption oracle specification relies on a verification algorithm, which is part of the FHE scheme and which takes as input a ciphertext to be decrypted and *a set of input ciphertexts* that explains the former (via some legit homomorphic evaluation). At this price, they achieve the tour de force of giving a construction that is IV-CCA secure in the standard model, based solely on the LWE assumption (and the additional assumption that *perfectly* correct FHE can be built from LWE⁸).

⁸ Indeed, [62] proposes an LWE-based construction instantiating the Naor-Yung paradigm, which explicitly requires *perfect* correctness from the underlying (FHE) scheme [36]. Although no explicit construction is provided in [62], their intent is to achieve this strong property by using truncated discrete gaussians for the noise distribution in order to get exact L_∞ norm bounds [61].

funcCPA security. FuncCPA is a recent security notion introduced in [1,2] in which the CPA game is extended with a functional re-encryption oracle in order to model setups in which a server performing computations over encrypted data, e.g. by means of FHE, is allowed to interactively delegate part of the computation back to the client, owner of the decryption key. More formally, given a family of functions $\mathcal{G}$, the adversary in the CPA experiment against (public-key) scheme $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ is further granted access to an additional oracle, denoted $\text{Recrypt}^{\mathcal{O}}$, which, given $g \in \mathcal{G}$ and $c \in \mathcal{C}$, returns,

$$\mathcal{E}.\text{Enc}(g(\mathcal{E}.\text{Dec}(c; \text{sk})); \text{pk}). \quad (5)$$

As a minor technical point, the above definition does not cover the case where the decryption function may return $\perp$. For simplicity sake, we stick to this simple definition and will further extend it when it will be later needed (consistently with [2, Remark 3]). Additionally, we also emphasize that [2] explicitly works under the correctness assumption of $\mathcal{E}$. Then, [2] establishes that funcCPA is strictly stronger than CPA (and, trivially, strictly weaker than CCA2) and provides several construction blueprints to turn CPA secure FHE schemes into funcCPA secure ones. In a follow up work, [35] further defines a variant with a multi-input oracle, denoted funcCPA^+ , which has recently been proved equivalent to the baseline notion [59].

With respect to weaker(-than-CCA2) CCA security notions, we further have the following result.

Corollary 1 (Theorem 4 in [50]). *CCA1 security implies funcCPA security.*

Proof. This is a corollary to [50, Theorem 4] which establishes that vCCA security implies funcCPA security. Indeed, that proof establishes this latter result by proving that CCA1 implies funcCPA (the final implication following from the fact that vCCA trivially implies CCA1). $\square$

This security notion is relevant here since funcCPA also models setups where, rather than delegating part of the encrypted domain computation all the way back to the client, the server has the ability to do this delegation towards a honest or semi-honest client proxy that it hosts, such as a secure enclave [35]. It is therefore our starting point in this paper as we now define and study funcCPA-style variants of the previously discussed CCA security notions.

In the sequel, we will thus consider the following generic security experiment.

Definition 2 (Generic security experiment).

$$\begin{aligned} (pk, sk) &\leftarrow \mathcal{E}.\text{KeyGen}(1^\lambda), \\ (m_0, m_1, S) &\leftarrow \mathcal{A}_1^{\text{Dec}_1^{\mathcal{O}}, \text{Func}_1^{\mathcal{O}}}(pk), \\ \gamma &\stackrel{\$}{\leftarrow} \{0, 1\}, \\ c^* &\leftarrow \mathcal{E}.\text{Enc}(m_\gamma), \\ b' &\leftarrow \mathcal{A}_2^{\text{Dec}_2^{\mathcal{O}}, \text{Func}_2^{\mathcal{O}}}(pk, S, c^*), \\ b &= b'. \end{aligned}$$

In this definition, the oracles $\text{Dec}_1^\mathcal{O}$, $\text{Dec}_2^\mathcal{O}$, $\text{Func}_1^\mathcal{O}$ and $\text{Func}_2^\mathcal{O}$ will vary depending on the security notion. For example, $\text{Dec}_1^\mathcal{O} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^\mathcal{O} = .$ and $\text{Func}_1^\mathcal{O} = \text{Func}_2^\mathcal{O} = .$ yields the CCA1 experiment, while $\text{Dec}_1^\mathcal{O} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^\mathcal{O} = \text{Dec}_\mathcal{O}^{\text{CCA}}$ (Eq. 4) and $\text{Func}_1^\mathcal{O} = \text{Func}_2^\mathcal{O} = .$ gives the vCCA experiment. Lastly, $\text{Dec}_1^\mathcal{O} = .$, $\text{Dec}_2^\mathcal{O} = .$ and $\text{Func}_1^\mathcal{O} = \text{Func}_2^\mathcal{O} = \text{Reencrypt}^\mathcal{O}$ yields the funcCPA experiment.

3 New variants of CCA1 security

3.1 A funcCPA-style variant

In this section we thus consider a first variant of CCA1 security where the adversary is further provided access to a reencrypt oracle, as in funcCPA (Eq. 5). We denote this notion by CCA1^R (i.e. $\text{Dec}_1^\mathcal{O} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^\mathcal{O} = .$ and $\text{Func}_1^\mathcal{O} = \text{Func}_2^\mathcal{O} = \text{Reencrypt}^\mathcal{O}$, for the generic experiment in Def. 2). We first establish that this new notion is no different from CCA1 itself in the next proposition. The structure of the first part of the proof is inspired from the part of the proof of Theorem 4 in [50] which establishes that funcCPA is equivalent to funcCPA^{*}⁹, still there are a number of differences.

Proposition 3. *CCA1^R is equivalent to CCA1.*

Proof. First, it is clear that CCA1^R implies CCA1. So we need to prove that CCA1 implies CCA1^R . Let $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec}, [\text{Eval}])$ denote a public-key (possibly homomorphic) encryption scheme. To ease the proof, we consider the modified experiment, $\text{CCA1}_\star^R$, in which a second phase (i.e. post-challenge) reencrypt request over ciphertext c is answered by $\mathcal{E}.\text{Enc}(m)$, for a randomly chosen plaintext m , rather than $\mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c))$ (for simplicity sake we omit the function that reencrypt may also evaluate).

Equivalence between CCA1^R and CCA1_★^R. Let us first prove that CCA1^R is equivalent to $\text{CCA1}_\star^R$. We denote by q the number of second phase reencrypt queries (numbered from 1 to q), and consider the following experiments:

- Hybrid H_q : is exactly the CCA1^R experiment.
- Hybrid H_j ($j = 0, \dots, q - 1$): proceeds similarly to H_q for handling second phase reencrypt requests $i \leq j$ and following $\text{CCA1}_\star^R$ when $i > j$.

Then, H_0 is the $\text{CCA1}_\star^R$ experiment. We now show that an adversary $\mathcal{B}$ against the CCA1 security of scheme $\mathcal{E}$ can be build from an adversary $\mathcal{A}$ able to distinguish H_{j-1} from H_j , for some $j \in [1, q]$. This is done by means of a reduction $\mathcal{B}$ which picks j^* at random and then handles $\mathcal{A}$'s requests as follows:

- First phase:
 - A decryption request from $\mathcal{A}$ is transferred as is to the CCA1 challenger.

⁹ FuncCPA^{*} is the variant of funcCPA with a second phase reencrypt oracle replying with an encryption of a random plaintext rather than a re-encryption of the submitted ciphertext.

- A decrypt request from $\mathcal{A}$ over ciphertext c is handled by sending a decryption request over c to the CCA1 challenger to get the corresponding message¹⁰ m and then returning $\mathcal{E}.\text{Enc}(m)$ to $\mathcal{A}$.
- A challenge request over $m_0 \neq m_1$ is handled by $\mathcal{B}$ picking $b \in \{0,1\}$ at random and returning $\mathcal{E}.\text{Enc}(m_b)$.
- Second phase: the second phase decrypt request number j is handled as follows:
 - If $j < j^*$, then $\mathcal{B}$ sends a decryption request over c to its CCA1 challenger to get the corresponding message m and then returns $\mathcal{E}.\text{Enc}(m)$ to $\mathcal{A}$ (thus following CCA1^R).
 - If $j = j^*$, then $\mathcal{B}$ first sends a decryption request over c to its CCA1 challenger to get the corresponding message m , it then picks m' at random and issues a challenge request with m and m' to its challenger to get ciphertext c' (which either encrypts m or m') and returns c' to $\mathcal{A}$ (i.e. following CCA1^R or CCA1_*^R depending on the challenger's secret bit).
 - If $j > j^*$, then $\mathcal{B}$ returns $\mathcal{E}.\text{Enc}(m)$ for a randomly picked m (thus following CCA1_*^R).

Hence, $\mathcal{B}$ simulates game H_{j^*} when the challenger encrypts m at request j^* , and game H_{j^*-1} when it encrypts a random message m' at this same request. The claim that CCA1_*^R is equivalent to CCA1^R thus follows from the standard hybrid argument.

Final reduction. This being done, we can now finalize the proof by showing that an adversary against the CCA1 security of $\mathcal{E}$ can be built by using an adversary $\mathcal{A}$ against its CCA1_*^R security. The reduction is straightforward:

- First phase decryption requests are transferred as is to the CCA1 challenger.
- A first phase decrypt request over ciphertext c is handled by sending a decryption request to the CCA1 challenger and returning an encryption of the result to $\mathcal{A}$.
- The challenge request is transferred as is to the CCA1 challenger.
- Finally, second phase decrypt requests are just handled by returning $\mathcal{E}.\text{Enc}(m)$ for a randomly picked message m .

□

Remark that the proof has no dependency on the correctness assumption and so the above result also holds for approximate or somewhat correct schemes.

3.2 A variant with a bivariate oracle

Given $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec}, [\text{Eval}])$, we now consider a variant of CCA1 security where the adversary is further provided access to an oracle performing a bivariate operation over the associated cleartexts of two ciphertexts. Considering multiplication as an example, we augment the CCA1 game with a multiplication oracle $\text{Mul}^{\mathcal{O}}$ which, given two encryptions of messages $m, m' \in \mathcal{M}$ returns

¹⁰ Or, more precisely, the corresponding decryption in case we do not work under the correctness assumption.

an encryption of mm' (or, more rigorously, an encryption of the product of their decryptions). In other words, given $c, c' \in \mathcal{C}$,

$$\text{Mul}^{\mathcal{O}}(c, c') = \mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c)\mathcal{E}.\text{Dec}(c')). \quad (6)$$

We denote this notion CCA1^M (for “CCA1 with a multiplication oracle”), i.e. $\text{Dec}_1^{\mathcal{O}} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^{\mathcal{O}} = \cdot$ and $\text{Func}_1^{\mathcal{O}} = \text{Func}_2^{\mathcal{O}} = \text{Mul}^{\mathcal{O}}$, for the generic experiment in Def. 2. We then have the following result.

Corollary 4. *CCA1^M is equivalent to CCA1.*

Proof. The proof is identical to that of the previous proposition to the exception that two decryptions requests have to be issued towards the CCA1 challenger when handling multiplication oracle requests (rather than only one to handle decrypt oracle requests), in the two reductions of that proof. $\square$

4 Leveraging on Paillier

In this section, as a warm-up on constructions, we work under the *heuristic* assumption that the Paillier scheme achieves CCA1 security. This assumption deserves some discussion. On the negative side, recent results [57,58] have revealed strong theoretical barriers against the provable CCA1 or even PCA1 security of the baseline scheme, by essentially establishing that no simple reduction treating the adversary as a black-box can establish its CCA1 security under a falsifiable assumption. On the brighter side, a CCA1 security proof exists in an idealized model¹¹ [4] and, from the cryptanalytic viewpoint, no CCA1 attack is known against the scheme (to the best of the authors’ knowledge). However, although it is clearly a non-falsifiable assumption (following Naor’s challenge-based categories of falsification [51]), we will now see that it yields a number of simple construction blueprints that we will eventually try to port over other security assumptions later in the paper.

We also highlight that the constructions in this section rely on the Paillier scheme in a black-box fashion. Hence, any correct CCA1 secure scheme which is linearly homomorphic without restriction (i.e. “fully” linearly homomorphic) may be used in lieu of Paillier. However, they cannot be instantiated as is when the base CCA1 secure scheme is only somewhat linearly homomorphic. To the best of our knowledge, all the linearly homomorphic schemes that are presently proven CCA1 secure under standard assumptions turns out to be only somewhat so (we will come back to this case and further discuss this assertion in the next Sect. 5 and, in particular, on p. 18). This further motivates our choice of using Paillier *as an example scheme* to instantiate the constructions in this section.

¹¹ A similar situation occurs for the ElGamal scheme which can be proved CCA1 secure in the idealized Generic as well as Algebraic Group Models [48,39] but is also covered by Schage’s result [57].

4.1 Building from vanilla Paillier

We now consider the Paillier scheme [54],

$$\mathcal{P} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Add}, \text{MulByPlain}, \text{AddPlain}),$$

recalled in appendix Sect. F, and associate it with a multiplication operator relying on a multiplication oracle $\text{Mul}^{\mathcal{O}}$ (following Eq. 6). Then, if we decide to believe that Paillier achieves CCA1 security, as discussed above, then Corr. 4 directly implies that $\mathcal{P}$ is CCA1^M secure.

Let n denote the RSA modulus of the scheme. Interestingly, we can specify the client-aided multiplication operator in such a way that the multiplication oracle can be implemented by means of an honest-but-curious rather than fully honest local client proxy. Indeed, we do so by applying a one-time mask to the messages, m and m' , associated with the two input ciphertexts by means of the scheme's AddPlain operator so that the oracle may observe only $[m + \mu]_n$ and $[m' + \mu']_n$. In exchange, the oracle generates an encryption of $(m + \mu)(m' + \mu') = mm' + \mu'm + \mu m' + \mu\mu'$ yet from which the caller can remove the spurious terms by means of the MulByPlain and AddPlain homomorphic operators to duly end up with an encryption of mm' . This is done below.

Given $c, c' \in \mathbb{Z}_{n^2}$ two encryptions of plaintext messages $m, m' \in \mathbb{Z}_n$:

- Pick masks μ, μ' uniformly in $\mathbb{Z}_n$.
- Invoke $\text{Mul}^{\mathcal{O}}$ over ciphertexts $[cg^{\mu}]_{n^2}$ and $[c'g^{\mu'}]_{n^2}$:
 - Then, $\text{Mul}^{\mathcal{O}}$ decrypts both ciphertexts to get $[m + \mu]_n$ and $[m' + \mu']_n$ and returns $c'' = \mathcal{P}.\text{Enc}([(m + \mu)(m' + \mu')]_n)$.
 - *Remark that $\text{Mul}^{\mathcal{O}}$ learns neither m nor m' in the process.*
- Then $c''c^{-\mu}c'^{-\mu'}g^{-\mu\mu'}$ is an encryption of mm' .

Alternative (with multiplicative rather than additive masking). To cope with an honest-but-curious multiplication oracle we can alternatively build a multiplication operator as follows: given $c, c' \in \mathbb{Z}_{n^2}$ two encryptions of $m, m' \in \mathbb{Z}_n$:

- Pick masks μ, μ' uniformly in $\mathbb{Z}_n^*$.
- Invoke $\text{Mul}^{\mathcal{O}}$ over ciphertexts $[c^{\mu}]_{n^2}$ and $[c'^{\mu'}]_{n^2}$:
 - Then, $\text{Mul}^{\mathcal{O}}$ decrypts both ciphertexts to get $[m\mu]_n$ and $[m'\mu']_n$ and returns $c'' = \mathcal{P}.\text{Enc}([m\mu m'\mu']_n)$.
 - *Remark that $\text{Mul}^{\mathcal{O}}$ learns neither m nor m' in the process.*
- Then $c''[(\mu\mu')^{-1}]_n$ is an encryption of mm' .

This latter alternative is more efficient than the previous one since the post $\text{Mul}^{\mathcal{O}}$ processing involves only one modular exponentiation. Of course, this works assuming that the probability that $\mu\mu'$ is not invertible in $\mathbb{Z}_n$ is $\text{neg}(\lambda)$, which is the case for Paillier (as n is the product of two large primes). Note also that we have expressed the above two oracles by “inlining” the Paillier scheme homomorphic operators but they can straightforwardly be expressed more generically by invoking MulByPlain and AddPlain .

4.2 Oracle-based relinearization of Catalano-Fiore ciphertexts

In [25], Catalano and Fiore have proposed a clever and simple trick to get one level of multiplication from Paillier, or any other linearly homomorphic scheme. In a nutshell, the trick consists in encrypting a message m as a pair $(c_0, c_1) = ([m - b]_n, \mathcal{P}.\text{Enc}(b))$ for a uniformly chosen mask $b \in \mathbb{Z}_n$. Then, given two such ciphertexts (c_0, c_1) and (c'_0, c'_1) , one may obtain an encryption of mm' by building a triplet of the form,

$$(\mathcal{P}.\text{Enc}(c_0 c'_0) c_1^{c'_0} c'_1^{c_0}, c_1, c'_1),$$

which first term is thus an encryption of $(m - b)(m' - b') + b(m' - b') + b'(m - b) = mm' - bb'$ under $\mathcal{P}$. Although still linearly homomorphic to some extent, the post-multiplication ciphertext structure is different from the pre-multiplication one and no procedure is known to switch from the latter form to the former, hence the limitation to one level of multiplication.

More formally, the Catalano-Fiore blueprint is specified as follows (assuming Paillier):

- CF.KeyGen: same as $\mathcal{P}.\text{KeyGen}$.
- CF.Enc: given $m \in \mathbb{Z}_n$, pick uniformly $b \in \mathbb{Z}_n$, return,

$$([m - b]_n, \mathcal{P}.\text{Enc}(b)) = (c_0, c_1).$$

- CF.Add: given two (first form) ciphertexts $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$ and $(c'_0, c'_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$ return (first form) ciphertext,

$$(c''_0, c''_1) = (c_0 + c'_0, c_1 c'_1).$$

- CF.Mul: given two (first form) ciphertexts $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$ and $(c'_0, c'_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$ return (second form) ciphertext,

$$(c''_0, c''_1, c''_2) = (\mathcal{P}.\text{Enc}(c_0 c'_0) c_1^{c'_0} c'_1^{c_0}, c_1, c'_1) \in \mathbb{Z}_{n^2}^3.$$

- CF.Dec₁ (first form): given $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$, return $[c_0 + \mathcal{P}.\text{Dec}(c_1)]_n$.
- CF.Dec₂ (second form): given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^3$, return,

$$[\mathcal{P}.\text{Dec}(c_0) + \mathcal{P}.\text{Dec}(c_1) \mathcal{P}.\text{Dec}(c_2)]_n.$$

Remark that the encryption function can equivalently be defined as:

- CF.Enc: given $m \in \mathbb{Z}_n$, pick uniformly $b \in \mathbb{Z}_n$, return,

$$(b, \mathcal{P}.\text{Enc}(m - b)) = (c_0, c_1). \quad (7)$$

This variant is more convenient for handling challenge requests in reductions, as we shall now see.

Proposition 5. *If there exists a CCA1 adversary $\mathcal{A}$ against CF, then there exists a CCA1 adversary $\mathcal{B}$ against $\mathcal{P}$ that uses $\mathcal{A}$ as a subroutine and achieves the same advantage.*

Proof. The proof is done via a simple reduction, in which $\mathcal{B}$ proceeds as follows, assuming the alternative encryption function in Eq. (7):

- When $\mathcal{A}$ issues a challenge request with messages $m_0, m_1 \in \mathbb{Z}_n$, then $\mathcal{B}$ uniformly picks $b \in \mathbb{Z}_n$ and issues a challenge request with $[m_0 - b]_n, [m_1 - b]_n$ towards the CCA1 challenger against $\mathcal{P}$ to get $c_1 = \mathcal{P}.\text{Enc}(m_\gamma - b)$ (for unknown bit γ). It then returns (b, c_1) to $\mathcal{A}$.
- When $\mathcal{A}$ issues a decryption request with (first form) ciphertext $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}$, $\mathcal{B}$ first issues a decryption request over c_1 towards its challenger to get μ and returns $[c_0 + \mu]_n$ to $\mathcal{A}$.
- When $\mathcal{A}$ issues a decryption request with (second form) ciphertext $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^3$, $\mathcal{B}$ issues decryption requests over c_0 , c_1 and c_2 towards its challenger to get μ_0 , μ_1 and μ_2 . It then returns $[\mu_0 + \mu_1\mu_2]_n$ to $\mathcal{A}$.

□

Remark that a ciphertext of the first form $c = (c_0, c_1)$ can be turned into a ciphertext of the second form c' by simply doing,

$$c' = \text{CF.Mul}(c, \text{CF.Enc}(1)).$$

When the trivial “encryption” of 1 under CF given by $(1 - 0, g^0 \bmod n^2) = (1, 1)$ is used, it suffices to do¹²,

$$c' = (\mathcal{P}.\text{Enc}(c_0)c_1, c_1, 1).$$

The corollary below then directly follows from Corr. 1 and Prop. 5.

Corollary 6. *CF is funcCPA secure.*

Let us now associate the CF scheme with a “relinearization” oracle¹³ $\text{Relin}^\mathcal{O}$ which, given a ciphertext of the second form, returns a fresh ciphertext of the first form encrypting the same message. In other words, given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^3$,

$$\begin{aligned} \text{Relin}^\mathcal{O}(c_0, c_1, c_2) &= \text{CF.Enc}(\text{CF.Dec}_2(c_0, c_1, c_2)), \\ &= \text{CF.Enc}([\mathcal{P}.\text{Dec}(c_0) + \mathcal{P}.\text{Dec}(c_1)\mathcal{P}.\text{Dec}(c_2)]_n) = (c'_0, c'_1). \end{aligned}$$

Then, consider the security notion obtained by further granting a CCA1 adversary access to a $\text{Relin}^\mathcal{O}$ oracle and denote this notion CCA1^L (for “CCA1 with a relinearization oracle”), i.e. $\text{Dec}_1^\mathcal{O} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^\mathcal{O} = .$ and $\text{Func}_1^\mathcal{O} = \text{Func}_2^\mathcal{O} = \text{Relin}^\mathcal{O}$, for the generic experiment in Def. 2. Since CF.Dec_1 and CF.Dec_2 can be unified under a single decryption function working over an ad hoc unified ciphertext form, e.g. given $(s, c_0, c_1, c_2, c_3) \in \{0, 1\} \times \mathbb{Z}_n \times \mathbb{Z}_{n^2} \times \mathbb{Z}_{n^2} \times \mathbb{Z}_{n^2}$, let,

$$\text{CF.Dec}(s, c_0, c_1, c_2, c_3) = \begin{cases} \text{CF.Dec}_1(c_0, c_1), & \text{if } s = 0, \\ \text{CF.Dec}_2(c_1, c_2, c_3), & \text{otherwise,} \end{cases}$$

¹² Because 1 acts as a trivial encryption of 0 under $\mathcal{P}$, doing $c' = (\mathcal{P}.\text{Enc}(c_0)c_1, 1, 1)$ also works.

¹³ The spirit is thus to account for schemes that can perform only one level of multiplication, yielding a different post-multiplication ciphertext structure that needs to be converted back to the pre-multiplication one to proceed with further calculations.

the above relinearization oracle is thus a special case of a reencrypt oracle, hence $\text{CCA1}^R \Rightarrow \text{CCA1}^L$. Since CCA1^L further trivially implies CCA1 and, provided that $\text{CCA1}^R \equiv \text{CCA1}$ (from Prop. 3), we have the following Corollary.

Corollary 7. *CCA1^L is equivalent to CCA1.*

Then, the CCA1 security of CF (Prop. 5) implies the following result.

Corollary 8. *CF is CCA1^L secure.*

Finally, to handle the case where the relinearization oracle is implemented by means of an honest-but-curious local client proxy, we can now build a relinearization operator that integrates one-time masking as follows. Given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^3$ a second form encryption of $m \in \mathbb{Z}_n$:

- Pick mask μ uniformly in $\mathbb{Z}_n$.
- Invoke $\text{Relin}^\mathcal{O}$ over ciphertext $(c_0 g^\mu, c_1, c_2)$:
 - Then, $\text{Relin}^\mathcal{O}$ decrypts that ciphertext to get $[m + \mu]_n$ and returns a first form ciphertext $c = \text{CF.Enc}([m + \mu]_n) = (c'_0, c'_1)$.
 - *Remark that $\text{Relin}^\mathcal{O}$ thus does not see m in the process.*
- Then $([c'_0 - \mu]_n, c'_1)$ is a first form encryption of m .

This gives us another way to achieve CCA1 secure client-aided “fully” homomorphic encryption, if one wishes to rely on a relinearization rather than a multiplication oracle (which may appear more natural).

5 Leveraging on Paillier-ElGamal

In this section, we abandon the heuristic assumption that Paillier is CCA1 secure and study the extent to which the constructs of the previous section can be ported over a variant of Paillier-ElGamal where λ plaintext bits are forced to be zeroes. This variant has recently been introduced in [47] and *shown CCA1 secure under the DCR assumption*¹⁴.

5.1 Paillier-ElGamal with plaintext zero padding

We now specify the Paillier-ElGamal scheme that is used as a starting point in [47]. This scheme originates in the Camenisch-Shoup scheme [23] (which is CCA2 under DCR and, as such, non-malleable) as it is essentially equivalent to a variant of the latter in which the last out of three slots is removed. We also stress that there are several CPA secure schemes referred to as Paillier-ElGamal in the literature, e.g. [53, 56, 21, 15], and they are not all equivalent.

So, following [47], the baseline Paillier-ElGamal scheme (which may thus also be referred to as the Camenisch-Shoup Lite scheme) that we use may be defined as follows:

¹⁴ Stricto sensu, it has been established under Decision Composite Residuosity (DCR) and an additional Composite Non-Invertibility assumption which is less standard but falsifiable. However, reliance on this latter assumption may be avoided under some conditions [47] to which we will come back.

- PEG.KeyGen: choose a RSA modulus $n = (2p' + 1)(2q' + 1)$, with p' and q' two distinct Sophie Germain primes of suitable size. Then, uniformly pick $g \in \mathbb{Z}_{n^2}^*$, $x \in \llbracket 0, n(n-1)/4 \rrbracket$ and let $h = [g^{4nx}]_{n^2}$. Let $\mathbf{pk} = (n, g, h)$ and $\mathbf{sk} = x$.
- PEG.Enc: given $m \in \mathbb{Z}_n$ and $\mathbf{pk}$, pick $r \in \llbracket 0, (n-1)/4 \rrbracket$ and return $(c_0, c_1) \in \mathbb{Z}_{n^2}^2$ such that,

$$c_0 = [g^{2nr}]_{n^2}, \text{ and } c_1 = [(1+n)^m h^r]_{n^2}.$$

- PEG.Dec: given $(c_0, c_1) \in \mathbb{Z}_{n^2}^2$ and $\mathbf{sk}$, let $\mu = [c_1 \cdot c_0^{-2x}]_{n^2}$. If μ is of the form $[(1+n)^m]_{n^2}$ for some $m \in \mathbb{Z}_n$, then return m and, otherwise, return $\perp$.

Remark that computing m in PEG.Dec may be done easily since $[(1+n)^m]_{n^2} = 1 + mn$. Hence, PEG.Dec can be equivalently written as returning $\perp$ when $[\mu - 1]_n \neq 0$ and $\frac{\mu-1}{n}$ otherwise.

This scheme is perfectly correct and trivially linearly homomorphic offering operators,

- PEG.Add: given (c_0, c_1) and (c'_0, c'_1) , both in $\mathbb{Z}_{n^2}^2$, return $([c_0 c'_0]_{n^2}, [c_1 c'_1]_{n^2})$.
- PEG.MulByPlain $_\alpha$: given $(c_0, c_1) \in \mathbb{Z}_{n^2}^2$ and $\alpha \in \mathbb{Z}_n$, return $([c_0^\alpha]_{n^2}, [c_1^\alpha]_{n^2})$.

It is further known that PEG is CPA secure under the DCR assumption.

In [47], the following variant of PEG is considered. Let $2^\lambda \leq B = 2^t < \lfloor 2^{-\lambda} n \rfloor$,

- PEG $_z$.KeyGen: PEG.KeyGen.
- PEG $_z$.Enc: given $m \in \llbracket 0, B \rrbracket$ return PEG.Enc(m).
- PEG $_z$.Dec: given $\text{ct} \in \mathbb{Z}_{n^2}^2$, if PEG.Dec(ct) = $\perp$, then return $\perp$. Otherwise, let $m = \text{PEG.Dec}(\text{ct})$, if $m > B$ then return $\perp$ and otherwise return m .

Remark that the resulting scheme *does not achieve perfect correctness anymore* (e.g. when a carry propagates into the zero padding during an homomorphic addition the resulting ciphertext will decrypt to $\perp$ rather than the sum of the plaintexts of the two input ciphertexts). For the same reason, it is also “only” *somewhat* linearly homomorphic (over $\mathbb{Z}_n$) which is a mild restriction in practice as the plaintext domain, $\mathbb{Z}_n$, is quite large. Note that, consistently with the above, PEG $_z$ ’s MulByPlain and AddPlain operators also take their plaintext argument in $\llbracket 0, B \rrbracket$ rather than $\mathbb{Z}_n$.

As a side remark, note that the restriction that the inputs of PEG $_z$.Enc are in $\llbracket 0, B \rrbracket$ is artificial since one may generate an encryption of *any* $m \in \mathbb{Z}_n$ by doing¹⁵,

$$\text{PEG}_z.\text{Add}(\text{PEG}_z.\text{MulByPlain}_B(\text{PEG}_z.\text{Enc}(\lfloor m/B \rfloor)), \text{PEG}_z.\text{Enc}(\lfloor m \rfloor_B)). \quad (8)$$

Still, such a ciphertext would decrypt to $\perp$ whenever $m > B$. Interestingly, we further prove in appendix Sect. C, that the scheme PEG' $_z$ obtained from PEG $_z$

¹⁵ As soon as $n \leq B^2$, which is usually the case since a natural choice to maximize the plaintext domain size is $B \approx 2^\lambda n$ with $2^\lambda \ll n$ for typical parameters. When this is not the case, and $n > m > B^2$, one may chain several invocations of MulByPlain following a technique given in [26, Sect. E] (we do not provide further details here, since this is a very minor point).

by replacing the encryption function with the one of PEG is also CCA1 secure, under the assumption that PEG_z is.

Following [47, Theorem 3], it is now known that PEG_z is CCA1 secure under the DCR assumption and an additional Composite Non-Invertibility assumption [47, Def. 3] (see also Appendix E). This latter assumption, which states that it is hard for an adversary to generate a *Paillier* encryption of a non-zero message which is not mutually prime to n , is less standard than DCR but has been around for several years (it can be traced back to [44, Assumption 3]) and falls clearly in the falsifiable category (still following the terminology in [51]). Furthermore, reliance on this latter assumption may be avoided under the condition that, [47, Sect. 5.3],

$$B < 2^{-\lambda} \min(p, q), \quad (9)$$

in which case PEG_z is CCA1 secure under the sole DCR assumption¹⁶. Further note that neither PEG nor PEG_z is covered by the impossibility results in [57, 58] as both schemes lack the required property that the validity of ciphertexts can be publicly verified¹⁷.

5.2 CCA1^R and CCA1^M security of PEG_z

Because PEG_z 's decryption function may return $\perp$, we first have to specify the behavior of the decrypt oracle whenever this occurs. The simplest way to do so is to have the oracle returning an invalid ciphertext in that case. Then, the decryption oracle may be defined as follows,

$$\text{Decrypt}^{\mathcal{O}}(c) = \begin{cases} \text{PEG}_z.\text{Enc}(\text{PEG}_z.\text{Dec}(c)), & \text{if } \text{PEG}_z.\text{Dec}(c) \neq \perp, \\ \text{PEG}_z.\text{Rerand}(c), & \text{otherwise,} \end{cases} \quad (10)$$

with $\text{PEG}_z.\text{Rerand}(c) = \text{PEG}_z.\text{Add}(c, \text{PEG}_z.\text{Enc}(0))$, thus yielding the desired property that $\text{PEG}_z.\text{Dec}(c) = \perp \Rightarrow \text{PEG}_z.\text{Dec}(\text{Decrypt}^{\mathcal{O}}(c)) = \perp$.

In a similar spirit, we can also define a multiplication oracle for PEG_z as follows,

$$\text{Mul}^{\mathcal{O}}(c, c') = \begin{cases} \text{PEG}_z.\text{MulByPlain}_{\text{PEG}_z.\text{Dec}(c)}(c'), & \text{if } \text{PEG}_z.\text{Dec}(c) \neq \perp, \\ \text{PEG}_z.\text{MulByPlain}_{\text{PEG}_z.\text{Dec}(c')}(c), & \text{if } \text{PEG}_z.\text{Dec}(c') \neq \perp, \\ \text{PEG}_z.\text{Rerand}(c), & \text{otherwise.} \end{cases} \quad (11)$$

Note that any oracle that can be done with the CCA1 decryption oracle of a challenger against PEG_z , as required by the reduction in the proof of Corr. 4, would be acceptable. So there are several possible alternatives. Still, the above definition ensures that the behavior of $\text{Mul}^{\mathcal{O}}$ is consistent to that of $\text{PEG}_z.\text{MulByPlain}$.

As already emphasized, recall that the proofs of Prop. 3 and Corr. 4 have no dependency on the correctness assumption and, hence, also holds for somewhat

¹⁶ The proof [47, Theorem 3] relies on CNI for hopping between games G3 and G4 [47, Lemma 4]. This game hop is not necessary when Eq. (10) holds [47, Sect. 5.3].

¹⁷ For the same reasons, Schage's result does not apply to the Damgaard-ElGamal scheme [33, 10] which is now proven CCA1 solely under the DDH assumption [47].

correct schemes such as PEG_z . The CCA1^R and CCA1^M security of PEG_z thus follows from these and its CCA1 security (which holds only under falsifiable assumptions and even only under DCR if condition 9 holds, recall footnote 16).

With PEG_z , it is however delicate to implement masking/unmasking prior and after invoking the multiplication oracle as we did in Sect. 4.1 when building from the Paillier scheme. This is due to the fact that the masks μ and μ' have to be drawn uniformly in $\mathbb{Z}_n$ and (homomorphically) added modulo n to the messages m and m' with the result of jeopardizing the λ leading zeroes condition (a similar observation applies to the variant with multiplicative masking). For the same reason, the Catalano-Fiore trick cannot be applied to PEG_z .

One way to work around this issue is to use the standard unchecked decryption function, PEG.Dec , within the multiplication oracle and the checked one, $\text{PEG}_z.\text{Dec}$, for genuine decryptions, i.e. to use,

$$\text{Mul}_H^{\mathcal{O}}(c, c') = \text{PEG.Enc}([\text{PEG.Dec}(c)\text{PEG.Dec}(c')]_n), \quad (12)$$

in lieu of Eq. (11). However, this is not consistent with the reductions in the proofs of Prop. 3 and Corr. 4 as these reductions have to emulate either a re-encrypt or a multiplicative oracle by means of their CCA1 challenger decryption oracle alone¹⁸ (which in this case implements $\text{PEG}_z.\text{Dec}$). Thus, we consider the above workaround as a heuristic and leave open the question of obtaining a construction based on PEG_z which would consistently combine masking/unmasking in the multiplication oracle and *provable* $\text{CCA1}^M/\text{CCA1}$ security (grounded in the CCA1 security of PEG_z only). Still, we speculate in appendix Sect. C that this heuristic may be sound under the additional non-falsifiable assumption that PEG_z (or rather PEG'_z , see above) is PA1 [10].

To conclude this section, we also emphasize that the problem of designing a *non-somewhat* linearly homomorphic scheme which is CCA1 secure under standard assumptions still remains an open question to the best of our knowledge. This question is interesting as it would allow us to avoid relying on the above heuristic to perform masking in the multiplication-oracle. We emphasize that schemes such as Dămgård-ElGamal [33,47] or Cramer-Shoup Lite [32], which are both proven CCA1 secure under DDH, are also only *somewhat* linearly homomorphic. This is so because their linearly additive variants requires solving a discrete-log in their decryption function, an operation that can be done efficiently only if a small upper bound is known on the value of the messages to decrypt.

6 Achieving vCCA security

We now turn our attention to the vCCA security notion recently introduced by Manulis and Nguyen [50]. When working under the correctness assump-

¹⁸ Furthermore, $\text{Mul}^{\mathcal{O}}$ and $\text{Mul}_H^{\mathcal{O}}$ are not equivalent. For example, if c and c' are encryptions of $n-1$ and $n-k$, $1 \leq k \leq B$, then $\text{PEG}_z.\text{Dec}(\text{Mul}^{\mathcal{O}}(c, c')) = \perp$ whereas $\text{PEG}_z.\text{Dec}(\text{Mul}_H^{\mathcal{O}}(c, c')) = k$ (since $[(n-1)(n-k)]_n = k \leq B$). Remark, that c and c' can be legitimately built by means of Eq. (8).

tion, which is (almost always) the case in this paper, this security notion is the strongest (known-so-far) CCA security notion strictly lying between CCA1 and CCA2 and yet achievable by homomorphic schemes.

In this section, we study constructions that achieve vCCA security under the CPA security of Paillier and the assumption that a previously known [12,40] “two-ciphertexts” blueprint which consists in sparsifying the ciphertext domain of a linearly homomorphic scheme¹⁹ (e.g. Paillier) satisfies the Linear-Only Homomorphism assumption. This assumption, that we formally recall in appendix Sect. A, states that the only way for building a valid ciphertext under a linearly (and no more) homomorphic scheme is to homomorphically evaluate a linear combination by applying the homomorphic operators of the scheme over ciphertexts that have been outputted by the encryption function. In exchange, the assumption materializes itself via the existence of an extractor which, given a valid ciphertext generated by the adversary as well as its trace of execution, retrieves the linear combination that “explains” the ciphertext, if any. As a distant sibling of D  mgard’s Knowledge of Exponent Assumption [33,10], the LOH assumption is *non falsifiable* [51].

Having said that, it turns out that the vCCA security notion is intimately connected to Proof-of-Knowledge approaches as the decryption oracles in its security game embeds a PPT witness extractor allowing to retrieve both the input ciphertexts and the function homomorphically applied when a well-formed evaluated ciphertext is submitted by the adversary. As such, the spirit of this notion is to model construction blueprints which embed proof material in their ciphertexts and rely on a SNARK to enforce genuine homomorphic evaluations over some well-formed input ciphertexts. In the realm of Proof-of-Knowledge, non-falsifiable assumptions, such as KEA (or variants) and LOH, are used in numerous works, e.g. [5,43,9,12,17,41,52,14]. This state of affairs is also explained by Gentry&Wichs’ no-go theorem [42] which establishes that no black-box reduction exists to prove a SNARG or SNARK construct secure based on a falsifiable assumption. This thus motivates the use of such assumptions when building Proof-of-Knowledge primitives. Because of the powerful witness extractor involved in the security notion definition, we may speculate that achieving vCCA security under falsifiable assumptions is also prevented by Gentry&Wichs’ result. So, using such assumptions to design vCCA secure schemes may be reasonable.

Lastly, we also highlight that the constructions in this section again rely on the Paillier scheme in a black-box fashion. Hence, any correct *CPA-only* secure scheme which is linear-only homomorphic without restriction (i.e. “fully” linear-only homomorphic) may be used in lieu of Paillier to instantiate them.

6.1 Variants of vCCA security with additional oracles

Similarly to what we have examined in the previous sections, we now first consider a variant of vCCA security where the adversary is further provided access

¹⁹ In a nutshell, encrypt a message m as a pair of ciphertexts $(\text{Enc}(m), \text{Enc}(\alpha \cdot m))$ for a secret multiplier α and check that the linear relation holds upon decryption.

to a reencrypt oracle, in the spirit of funcCPA. We refer to this security notion by vCCA^R . Again, we have to specify the behavior of the reencrypt oracle when the decryption function returns $\perp$, so let us now clarify this point here. Given a linearly homomorphic scheme,

$$\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Add}, \text{MulByPlain}, \text{Rerand}),$$

we consider the following reencrypt oracle,

$$\text{Reencrypt}^{\mathcal{O}}(c) = \begin{cases} \mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c)), & \text{if } \mathcal{E}.\text{Dec}(c) \neq \perp, \\ \mathcal{E}.\text{Rerand}(c), & \text{otherwise,} \end{cases} \quad (13)$$

where $\mathcal{E}.\text{Rerand}(c) = \mathcal{E}.\text{Add}(c, \mathcal{E}.\text{Enc}(0))$. Note that the above definition is consistent with Remark 3 of [2]. Additionally, we work again under the natural assumption that $\mathcal{E}.\text{Dec}(c) = \perp$ implies that $\mathcal{E}.\text{Dec}(\mathcal{E}.\text{Rerand}(c))$ also returns $\perp$. Consistently with the vCCA game decryption oracle (recall the definition in Sect. 2.2, p. 6) vCCA^R has different first step and second step reencrypt oracles. While the first step oracle unconditionally follows Eq. (13), the second step reencrypt oracle, which we refer to as $\text{Reencrypt}_{\text{vCCA}}^{\mathcal{O}}$, proceeds as follows,

- Recryption request (2nd step): when $\mathcal{A}$ queries $(\text{reencrypt}, c)$ proceed as follows. Let $(f, c_0, \dots, c_{L-1}) = \text{Extract}(c, \text{aux})$. Then, return $\mathcal{E}.\text{Rerand}(c^*)$ when $c^* \in \{c_0, \dots, c_{L-1}\}$ and $\text{Reencrypt}^{\mathcal{O}}(c)$ otherwise.

Defining this second step reencrypt oracle avoids the pitfall of an empty definition. Indeed, without this modification, an adversary would always be able to perform the trivial attack consisting of asking the decryption of a re-encryption of the challenge ciphertext, c^* . Recall from Sect. 2.2 that $\text{Extract}(c, \text{aux}) = (\text{id}, c)$ whenever $c \in \mathcal{C}$ has *not* been obtained by invoking Eval over some function f and a set of ciphertexts. This is the case for a ciphertext of the form $\mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c^*))$. Following this, for the generic experiment in Def. 2, vCCA^R corresponds to $\text{Dec}_1^{\mathcal{O}} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^{\mathcal{O}} = \text{Dec}_{\text{vCCA}}^{\mathcal{O}}$ (Eq. 4), $\text{Func}_1^{\mathcal{O}} = \text{Reencrypt}^{\mathcal{O}}$ and $\text{Func}_2^{\mathcal{O}} = \text{Reencrypt}_{\text{vCCA}}^{\mathcal{O}}$.

We now show that vCCA^R and vCCA security are equivalent.

Proposition 9. *vCCA^R and vCCA security are equivalent.*

Proof. First, vCCA^R trivially implies vCCA . So we focus on proving that vCCA implies vCCA^R . Let $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Rerand})$ denote a correct encryption scheme. Then we show that from an adversary $\mathcal{A}$ against the vCCA^R security of $\mathcal{E}$, it is possible to build an adversary $\mathcal{B}$ against the vCCA security of $\mathcal{E}$ with the same advantage. The reduction $\mathcal{B}$ then proceeds as follows:

- Challenge and decryption requests are forwarded as is to the vCCA challenger.
- To handle a reencrypt request over ciphertext c , $\mathcal{B}$ first issues a decryption request toward its vCCA challenger to get either $\perp$, in which case it runs $\mathcal{E}.\text{Rerand}(c)$ on its own and returns the result to $\mathcal{A}$, or some value $r \in \mathcal{M}$, in which case it runs $\mathcal{E}.\text{Enc}(r)$ on its own and returns the result to $\mathcal{A}$.

The reduction works since, when handling a decrypt request from $\mathcal{A}$, in the two cases where the challenger may return $\perp$ to a decryption request (either the submitted ciphertext c decrypts to $\perp$ or is challenge dependent i.e. $c^* \in \{c_0, \dots, c_{L-1}\}$ with $(f, c_0, \dots, c_{L-1}) = \text{Extract}(c, \text{aux})$) the reduction replies with a rerandomization of the submitted ciphertext, consistently with the above definitions. $\square$

In a similar fashion, we now define vCCA^M security in which the vCCA adversary is provided with a multiplication oracle $\text{Mul}^\mathcal{O}(c, c')$ such that,

$$\text{Mul}^\mathcal{O}(c, c') = \begin{cases} \mathcal{E}.\text{Rerand}(c) & \text{if } \mathcal{E}.\text{Dec}(c) = \perp, \\ \mathcal{E}.\text{Rerand}(c') & \text{if } \mathcal{E}.\text{Dec}(c') = \perp, \\ \mathcal{E}.\text{Enc}(\mathcal{E}.\text{Dec}(c)\mathcal{E}.\text{Dec}(c')) & \text{otherwise.} \end{cases} \quad (14)$$

Then we define vCCA^M second step multiplication oracle ($\text{Mul}_{\text{vCCA}}^\mathcal{O}$) as:

- Multiplication request (2nd step): when $\mathcal{A}$ queries (mul, c, c') proceed as follows:
 - Let $(f, c_0, \dots, c_{L-1}) = \text{Extract}(c, \text{aux})$, then, if $c^* \in \{c_0, \dots, c_{L-1}\}$, return $\mathcal{E}.\text{Rerand}(c^*)$.
 - Let $(f', c'_0, \dots, c'_{L-1}) = \text{Extract}(c', \text{aux})$, then, if $c^* \in \{c'_0, \dots, c'_{L-1}\}$, return $\mathcal{E}.\text{Rerand}(c^*)$.
 - Otherwise, return $\text{Mul}^\mathcal{O}(c, c')$.

Hence, for the generic experiment in Def. 2, vCCA^R corresponds to $\text{Dec}_1^\mathcal{O} = \mathcal{E}.\text{Dec}$, $\text{Dec}_2^\mathcal{O} = \text{Dec}_{\text{vCCA}}^\mathcal{O}$ (Eq. 4), $\text{Func}_1^\mathcal{O} = \text{Mul}^\mathcal{O}$ and $\text{Func}_2^\mathcal{O} = \text{Mul}_{\text{vCCA}}^\mathcal{O}$. With this definition we have the straightforward corollary to Prop. 9.

Corollary 10. *vCCA^M and vCCA security are equivalent.*

6.2 A concrete construction based on a KEA extension of Paillier

We now consider a scheme which is built from the Paillier scheme by following a “two-ciphertexts” blueprint which consists in sparsifying the ciphertext domain of the base scheme by following a Knowledge-of-Exponent (KEA) template²⁰ [33]. In a nutshell, the encryption of a message m consists of a pair of ciphertexts $(\text{Enc}(m), \text{Enc}(\xi \cdot m))$ under the same key material, for a secret random value $\xi \in \mathbb{Z}_n$, and with an image verification algorithm, ImVer , checking this linear relation by decrypting both ciphertexts. We emphasize that this scheme is not new and has already been proposed and used in several works, e.g. [12, 40, 26]. This kind of schemes are also assumed to satisfy the Linear-Only Homomorphism (LOH) assumption that we recall in details in appendix Sect. A. Note also that in the notations of Def. 15, App. A, ImVer is a function that tests, using the secret key, whether a given candidate ciphertext is in the image of the encryption function of a given scheme. More formally, we now consider the scheme $\mathcal{P}^{(2)}$ which is build from $\mathcal{P}$ as follows:

²⁰ This is a slight abuse of terminology. Yet, we will use it in this paper as the “two-ciphertexts” blueprint is a distant sibling of D  mgard original heuristic [33] and similar in spirit.

- $\mathcal{P}^{(2)}.\text{KeyGen}$: run $\mathcal{P}.\text{KeyGen}$ to get n , g and ω , then pick ξ uniformly at random in $\mathbb{Z}_n$. Generate ciphertext $\text{ct}^\Delta = (\mathcal{P}.\text{Enc}(1), \mathcal{P}.\text{Enc}(\xi))$. The public key is formed by n and ct^Δ , while all the other parameters remain private.
- $\mathcal{P}^{(2)}.\text{Enc}$: given $m \in \mathbb{Z}_n$ pick (r_0, r_1) uniformly in $\mathbb{Z}_n^2$ and return,

$$\text{ct} = (c_0, c_1) = ([(\text{ct}^\Delta.c_0)^m r_0^n]_{n^2}, [(\text{ct}^\Delta.c_1)^m r_1^n]_{n^2}). \quad (15)$$

- $\mathcal{P}^{(2)}.\text{ImVer}$: given $\text{ct} \in \mathbb{Z}_{n^2}^2$, let μ_0 denote $\mathcal{P}.\text{Dec}(\text{ct}.c_0)$ (and respectively so for μ_1). Then return **True** if $[\xi\mu_0]_n = \mu_1$ and **False** otherwise.
- $\mathcal{P}^{(2)}.\text{Dec}$: given $\text{ct} \in \mathbb{Z}_{n^2}^2$, if $\mathcal{P}^{(2)}.\text{ImVer}(\text{ct}) = \text{True}$, return $\mathcal{P}.\text{Dec}(\text{ct}.c_0)$. Otherwise, return $\perp$.

Lastly, $\mathcal{P}^{(2)}.\text{Add}$ and $\mathcal{P}^{(2)}.\text{MulByPlain}$ operators are straightforwardly derived from those of $\mathcal{P}$. Remark, however, that $\mathcal{P}^{(2)}$ does not offer a direct add-by-const operator since the multiplier ξ as well as the group generator g are private²¹. To homomorphically add a constant $v \in \mathbb{Z}_n$, one should then explicitly call the encryption function, i.e.

$$\mathcal{P}^{(2)}.\text{AddPlain}(c, v) \equiv \mathcal{P}^{(2)}.\text{Add}(c, \mathcal{P}^{(2)}.\text{Enc}(v)). \quad (16)$$

Of course, $\mathcal{P}^{(2)}$ is only linearly homomorphic.

We then have the following proposition which is proved in [26]. For self-containedness, we provide an alternative direct proof in appendix Sect. B.

Proposition 11. *$\mathcal{P}^{(2)}$ is vCCA secure under the assumption that it has the LOH property and that $\mathcal{P}$ is CPA secure.*

Additionally, it is remarkable that such a simple construction may be proven to achieve vCCA security (and consequently also CCA1 security) at only twice the cost of the base Paillier scheme (i.e. of just achieving CPA security). Also, it may be worth emphasizing that the CCA1 security of D  mgard-ElGamal has recently been proven *solely* under the DDH assumption [47]. Although this does not easily carry over to the present two-ciphertexts heuristic, this may be interpreted as a hint that such constructions may have stronger foundations than previously considered.

Corollary 10 and Prop. 11 then directly yield the following result.

Proposition 12. *$\mathcal{P}^{(2)}$ is vCCA^M secure.*

Remark that the two multiplicative oracle flavors (with either additive or multiplicative masking) that we defined in Sect. 4.1 are also applicable to $\mathcal{P}^{(2)}$. The only subtlety is that Eq. (16) should be used (rather a direct add-by-const operator) to implement the variant with additive masking. Additionally, the case where $\mathcal{P}^{(2)}.\text{Dec}(c)$ returns $\perp$ in the oracle has to be consistently handled following Eq. (13).

²¹ The group generator is kept private to ensure that only encryptions of 0 under $\mathcal{P}^{(2)}$ can be generated without using ct^Δ . This condition is necessary to obtain the plaintext extractor needed by the proof of Prop. 11 (App. B, p. 28) and that of [26, Prop. 2]. See [26, Sect. 3.2] for further details.

6.3 Another Catalano-Fiore-based variant

In the spirit of the previous construction, we now instantiate the Catalano-Fiore blueprint of Sect. 4.2 but using this time $\mathcal{P}^{(2)}$ as the base scheme (again, considering the alternative encryption function in Eq. 7). Because this is done quite similarly to the instantiation of CF from $\mathcal{P}$, we defer the full definition of the scheme to Appendix D and focus here only on the ImVer and Dec functions.

- $\text{CF}^{(2)}.\text{ImVer}_1$ (first form): given $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$, return $\mathcal{P}^{(2)}.\text{ImVer}(c_1)$.
- $\text{CF}^{(2)}.\text{ImVer}_2$ (second form): given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, return **True** if $\mathcal{P}^{(2)}.\text{ImVer}(c_i) = \text{True}$ for all $i \in \{0, 1, 2\}$.
- $\text{CF}^{(2)}.\text{Dec}_1$ (first form): given $\text{ct} = (c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$, return $[c_0 + \mathcal{P}^{(2)}.\text{Dec}(c_1)]_n$ if $\text{CF}^{(2)}.\text{ImVer}_1(\text{ct}) = \text{True}$ and $\perp$ otherwise.
- $\text{CF}^{(2)}.\text{Dec}_2$ (second form): given $\text{ct} = (c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, return,

$$[\mathcal{P}^{(2)}.\text{Dec}(c_0) + \mathcal{P}^{(2)}.\text{Dec}(c_1)\mathcal{P}^{(2)}.\text{Dec}(c_2)]_n,$$

if $\text{CF}^{(2)}.\text{ImVer}_2(\text{ct}) = \text{True}$ and $\perp$ otherwise.

We then show that $\text{CF}^{(2)}$ achieves vCCA security under the assumption that $\mathcal{P}^{(2)}$ does (Prop. 11).

Proposition 13. *If there exists a vCCA adversary $\mathcal{A}$ against $\text{CF}^{(2)}$ then there exists a vCCA adversary $\mathcal{B}$ against $\mathcal{P}^{(2)}$ that uses $\mathcal{A}$ as a subroutine and achieves the same advantage.*

Proof. The proof is done via a simple reduction, which is quite similar to the one in the proof of Prop. 5, in which $\mathcal{B}$ proceeds as follows:

- When $\mathcal{A}$ issues a challenge request with messages $m_0, m_1 \in \mathbb{Z}_n$, then $\mathcal{B}$ uniformly picks $b \in \mathbb{Z}_n$ and further issues a challenge request with $[m_0 - b]_n, [m_1 - b]_n$ towards the vCCA challenger against $\mathcal{P}^{(2)}$ to get $c_1 = \mathcal{P}^{(2)}.\text{Enc}(m_\gamma - b)$ (for unknown bit γ). It then returns (b, c_1) to $\mathcal{A}$.
- When $\mathcal{A}$ issues a decryption request with (first form) ciphertext $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$, $\mathcal{B}$ first issues a decryption request over c_1 towards its challenger to either get $\perp$, in which case it also returns $\perp$ to $\mathcal{A}$, or $\mu \neq \perp$ and returns $[c_0 + \mu]_n$ to $\mathcal{A}$.
- When $\mathcal{A}$ issues a decryption request with (second form) ciphertext $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, $\mathcal{B}$ issues decryption requests over c_0, c_1 and c_2 towards its challenger. If at least one of these requests is replied to with $\perp$, it also returns the same to $\mathcal{A}$. Otherwise, $\mathcal{B}$ gets μ_0, μ_1 and μ_2 (all non- $\perp$) and then returns $[\mu_0 + \mu_1\mu_2]_n$ to $\mathcal{A}$.

□

To the best of our knowledge, $\text{CF}^{(2)}$ is then the first concrete scheme supporting both homomorphic additions and multiplications (even limited to one-level) that is proven vCCA secure.

Similarly to Sect. 4.2 and consistently with Eq. (13), we can then consider a relinearization oracle which, given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, returns,

$$\text{Relin}^{\mathcal{O}}(c_0, c_1, c_2) = \begin{cases} \text{CF}^{(2)}. \text{Enc}(\text{CF}^{(2)}. \text{Dec}_2(c_0, c_1, c_2)), & \text{if } \mathcal{P}^{(2)}. \text{Dec}(c_i) \neq \perp, \forall i \in \llbracket 0, 2 \rrbracket, \\ (b, \mathcal{P}^{(2)}. \text{Rerand}(c_i)), & \text{if } \exists i \in \llbracket 0, 2 \rrbracket : \mathcal{P}^{(2)}. \text{Dec}(c_i) = \perp, \end{cases} \quad (17)$$

with b picked uniformly in $\mathbb{Z}_n$, and consider the security notion, vCCA^L (for “vCCA with a relinearization oracle”), obtained by further granting a vCCA adversary access to a $\text{Relin}^{\mathcal{O}}$ oracle which is a special case of a decrypt oracle. Since we trivially have $\text{vCCA} \Leftarrow \text{vCCA}^L \Leftarrow \text{vCCA}^R$ and since (from Prop. 9) $\text{vCCA} \equiv \text{vCCA}^R$, the next result follows from Prop. 13.

Corollary 14. $\text{CF}^{(2)}$ is vCCA^L secure.

As a last remark, recall that $\mathcal{P}^{(2)}$ does not offer a direct addition-by-constant operator. Still, to cope with a relinearization oracle instantiated in an honest-but-curious client proxy, we can build a relinearization operator as follows, given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$ a second form encryption of $m \in \mathbb{Z}_n$ under $\text{CF}^{(2)}$:

- Pick mask μ uniformly in $\mathbb{Z}_n$.
- Invoke $\text{Relin}^{\mathcal{O}}$ over ciphertext $(\text{CF}^{(2)}. \text{Add}(c_0, \text{CF}^{(2)}. \text{Enc}(\mu)), c_1, c_2)$:
 - Then, $\text{Relin}^{\mathcal{O}}$ proceeds following Eq. (17) over this ciphertext.
 - Remark that $\text{Relin}^{\mathcal{O}}$ sees either $[m + \mu]_n$ or $\perp$ in the process and therefore does not learn m in the former case.
- Then, correspondingly, $(c'_0, \text{CF}^{(2)}. \text{Add}(c'_1, \text{CF}^{(2)}. \text{Enc}(-\mu)))$ is either a first form encryption of m or a ciphertext that decrypts to $\perp$.

We then end up with an additional way to achieve vCCA secure client-aided “fully” homomorphic encryption relying on a relinearization rather than a multiplication oracle.

7 Concluding remarks

Overall, the results in this paper reveal that “old school” number-theoretic linearly homomorphic encryption schemes may be meaningfully combined with secure enclaves (as faithful implementations of non-colluding semi-honest client proxies hosted on the entity performing the encrypted-domain calculations) in simple and pragmatic constructions in order to both achieve advanced CCA security properties and FHE-like capabilities. Because these schemes are known to induce relatively moderate computational burden and ciphertext sizes, the constructions presented in this paper may have a practical interest beyond their improved security properties [38]. We emphasize that lattice-based FHE presently offer no practical alternative for achieving CCA security properties, even based on heuristics. Although some theoretical works indicate that this may be possible, all present lattice-based CCA-secure FHE constructions are only of theoretical interest. In contrast, we claim that our constructions may lead to practical

Scheme	Sect.	Add.	Mul.	Sec. level	Hyp.
$\mathcal{P}$	4.1	Y	N	CCA1/CCA1 ^M	Heuristic
CF	4.2	Y	Y*	CCA1/CCA1 ^L	Heuristic
PEG _z	5.1	Y*	N	CCA1/CCA1 ^M	DCR+CNI
PEG _z	Eq. (9)	Y*	N	CCA1/CCA1 ^M	DCR
$\mathcal{P}^{(2)}$	6.2	Y	N	vCCA/vCCA ^M	DCR+LOH
CF ⁽²⁾	6.3	Y	Y*	vCCA/vCCA ^L	DCR+LOH

Table 1. Summary of our constructions. Column “Sect.” indicates the Section in which it is defined, “Add.” and “Mul.” whether it is additively or multiplicatively homomorphic (a “Y*” indicates that it is only somewhat so, see the respective sections for details). Then, columns “Sec. level” and “Hyp.” respectively indicate the security notion achieved and under which assumptions it is so.

alternatives for achieving CCA security. However, we do not claim that our constructions may outperform CPA-secure lattice-based FHE.

Additionally, the CCA security properties of our construction are achieved under several assumptions, some of them heuristic or non-falsifiable and some of them more standard (e.g. the CCA1 security of Libert’s zero-padded variant of Paillier-ElGamal), which brings some degree of diversity. We summarize this state of affairs in Table 1. Also note that other (correct) linearly homomorphic schemes can be plugged in the constructions presented in this paper. However, as already hinted at, it seems difficult to use the Dămgard-ElGamal [33,47] or Cramer-Shoup-Lite schemes [32] (for both of which CCA1 security holds under standard assumptions) as the additive variants of these schemes require solving a discrete-log in the decryption function, so their effective plaintext size is limited by this constraint²². This makes it difficult to use them in combination with the masking techniques we used to implement our semi-honest oracles, or to apply the Catalone-Fiore blueprint from them.

An interesting next step is to embed these constructions in more complete protocols including transmission of data. For example, a semi-honest local proxy may also help to perform transciphering for uplink (client to server) and downlink (server to client) ciphertexts transmission²³. With enclave-aided transciphering, one may convert data sent by the client and encrypted using a symmetric encryption scheme such as AES into HE form without paying the cost of an expensive homomorphic execution of the symmetric scheme decryption function [6]. Informally, with such an approach, a client may synchronize over a PRF with the server and share an AES key with a secure enclave, the client would then send $c = \text{AES.Enc}(m + \mu)$, for a one-time mask μ generated from the PRF, to the server/enclave pair with the enclave computing $c' = \text{HE.Enc}(\text{AES.Dec}(c))$ and

²² Furthermore, as a consequence, these schemes are “only” somewhat linearly homomorphic and do not satisfy correctness (recall Eq. 2).

²³ It should however be emphasized that while the uplink direction can be dealt with by running the symmetric algorithm decryption function over the target homomorphic scheme, the downlink direction is much more delicate to handle [15].

the server eventually doing $\text{HE.AddPlain}(c', -\mu)$ to end up with an encryption of m under HE as a starting point for an enclave-assisted encrypted domain calculation.

While the security properties of such folklore protocols remain to be studied (while also considering more concrete threat models for enclaves), this paper’s constructions may thus contribute leading to end-to-end protocols with efficient communications in which a client would rely on the association of a possibly malicious server with a non-colluding semi-honest enclave to perform distant encrypted domain computations. Another interesting observation, protocol-wise, is that number-theoretic schemes such as Paillier natively achieve properties such as circuit-privacy.

Lastly, an additional set of interesting questions is to investigate whether LWE or RLWE-based schemes may also benefit from this kind of approaches in order to also achieve some degree of “beyond CPA” security. We think that answering this question is not straightforward as these schemes are notoriously prone to decryption errors which may induce leakage on ciphertext noises, eventually leading to some attacks in the CPA^D model [27,28]. One way to achieve CPA^D security is to achieve correctness [45,27]. This can for example be done by systematically performing a (masked) re-encryption operation within a local client proxy as a substitute for bootstrapping after each operation (with suitably chosen parameters). Thus, while constructions such as those discussed in this paper (using either or both decrypt or relinearization oracles over a LWE or RLWE-base schemes) can credibly allow to achieve this level of security, it should be emphasized that CPA^D security is still a “passive” security model in which the adversary is supposed to play by the book relative to the cryptosystem specification. So such constructions will not help much when opening the Pandora box of CCA decryption oracles accepting arbitrary ciphertexts and it is likely that additional cryptographic tools have to be onboarded in the spirit of vCCA or vCCA^D generic construction blueprints [50,22]. The hope is then that relying on a semi-honest local client proxy may help lessening the complexity of the additional tools involved.

A The Linear-Only Homomorphism (LOH) assumption

Informally, for an encryption scheme $\mathcal{E}_H = (\text{KeyGen}, \text{Enc}, \text{ImVer}, \text{Dec}, \text{Eval})$, the Linear-Only Homomorphism (LOH) property states (as explained in [12]) that given polynomially-many ciphertexts $(c_0, \dots, c_{m-1})$ under $\mathcal{E}_H$, it is infeasible for an adversary to create a new ciphertext c' which is in the image of the encryption function (as verified by ImVer) and cannot be expressed by (homomorphically) evaluating an affine combination of the ciphertexts in the previous list. In the above, ImVer is a function that verifies if a ciphertext is in the image of the encryption function with knowledge of sk (it is essentially the “verification” part of $\mathcal{E}_H.\text{Dec}$ and $\mathcal{E}_H.\text{ImVer}(c) = \text{True} \Leftrightarrow \mathcal{E}_H.\text{Dec}(c) \neq \perp$). The LOH property has been introduced in [12] to serve as the basis for several SNARK constructions in that paper and other subsequent works [17,41,52].

Formally, following [12,13], we have the following definition.

Definition 15 (LOH property, reproduced from [12]). *An encryption scheme $\mathcal{E}_H = (\text{KeyGen}, \text{Enc}, \text{ImVer}, \text{Dec}, \text{Eval})$ (with $\mathcal{M} = \mathbb{Z}_t^{24}$) satisfies the Linear-only Homomorphism property if for every PPT adversary $\mathcal{A}$, there is a PPT extractor Extract such that for any auxilliary input $\text{aux} \in \{0,1\}^{\text{poly}(\lambda)}$ and any plaintext generator $\mathcal{M}$,*

$$P \left(\begin{array}{l} \exists i \in \llbracket 0, k-1 \rrbracket \text{ s. t.} \\ \text{ImVer}(c'_i) = \text{True} \\ \text{and} \\ \mathcal{E}_H.\text{Dec}(c'_i) \neq a'_i \end{array} \middle| \begin{array}{l} (ek, sk) := \mathcal{E}_H.\text{KeyGen}(1^\lambda) \\ (a_0, \dots, a_{m-1}) := \mathcal{M}([ek]) \\ (c_0, \dots, c_{m-1}) := (\mathcal{E}_H.\text{Enc}(a_0), \dots, \mathcal{E}_H.\text{Enc}(a_{m-1})) \\ (c'_0, \dots, c'_{k-1}) := \mathcal{A}(c_0, \dots, c_{m-1}, [ek]; \text{aux}) \\ (\Pi, b) := \text{Extract}(c_0, \dots, c_{m-1}, [ek]; \text{aux}) \\ (a'_0, \dots, a'_{k-1})^T := \Pi \cdot (a_0, \dots, a_{m-1})^T + b^T \end{array} \right) \leq \text{neg}(\lambda). \quad (18)$$

where $\Pi \in \mathbb{Z}_t^{k \times m}$ and $b \in \mathbb{Z}_t^k$, and with the convention that row i of Π is left empty (i.e. $(\Pi_i, b_i) = \emptyset$) and $a'_i = \perp$ when c'_i was not generated by homomorphically evaluating an affine combination over the c_j 's. The notation $[ek]$ indicates that the encryption key ek is optionally provided, depending on whether the setting is private or public-key.

Remark that the c'_i do not have to be explicitly included as inputs to the extractor since they are built by the adversary and therefore implicitly included in aux . In summary, whenever $\mathcal{A}$ builds c'_i by doing “something equivalent to”,

$$c'_i = \mathcal{E}_H.\text{Add}(\mathcal{E}_H.\text{Eval}(\text{lincomb}_\pi, c_0, \dots, c_{m-1}), \underbrace{\mathcal{E}_H.\text{Eval}(\text{lincomb}_{\pi'}, c''_0, \dots, c''_{l-1})}_{\text{pub. key case only}}), \quad (19)$$

where $c''_0 = \mathcal{E}_H.\text{Enc}(\mu_0; \text{pk}), \dots, c''_{l-1} = \mathcal{E}_H.\text{Enc}(\mu_{l-1}; \text{pk})$, then $\Pi_i = \pi$ and $b_i = \sum_{j=0}^{l-1} \pi'_j \mu_j$. Conversely, $(\Pi_i, b_i) = \emptyset$ when this is not the case.

In our security proofs, we will use the following more convenient one-ciphertext notation for the above extractor,

$$(\pi, \beta) = \text{Extract}(c, \text{aux}), \quad (20)$$

as a shortcut for $\Pi_{i:c'_i=c}$ and $b_{i:c'_i=c}$ (i.e. π , respectively β , is the row of Π , respectively the component of b , associated with ciphertext c). While a more rigorous notation would be to use $(\pi, \beta) = \text{Extract}(c, \{c_0, \dots, c_{m-1}\}, \text{aux})$, we use Eq. (20) in the sequel as the set of ciphertexts given to the LOH adversary will always be clear from the nearby context. Remark that affine rather than linear combinations are considered in the above definition to account for the fact that, in the public-key setting, the adversary can create (from scratch) additional fresh well-formed ciphertexts on its own (a case that does not happen in the private-key setting) and homomorphically add them to homomorphic evaluations of linear combinations over the c_i 's.

²⁴ The definition still extends to the case where the plaintext domain is a polynomial ring [12].

On top of the above definition, [12] further proposes several heuristic approaches to build schemes satisfying the LOH property, starting from a *correct* CPA secure linear homomorphic scheme. An example, which has been considered in several works (e.g. [12,40]), is the “two-ciphertexts” blueprint which consists in sparsifying the ciphertext domain of a correct linearly homomorphic scheme (e.g. Paillier) by following a Knowledge-of-Exponent (KEA) template²⁵. To illustrate this approach with the Paillier scheme, one may consider that the encryption of a message m consists of a pair of ciphertexts $(\text{Enc}(m), \text{Enc}(\alpha \cdot m))$ under the same key material, for a secret random value $\alpha \in \mathbb{Z}_n$ (with n the RSA modulus of the scheme), and with the `ImVer` algorithm checking this linear relation by decrypting both ciphertexts. It is then assumed that this scheme satisfies Definition 15 as the underlying Paillier scheme only exhibits linear homomorphic properties (to the best of the research community’s knowledge), although the baseline Paillier scheme does not have the LOH property as also argued in [12] ([11], p. 33). Interestingly, this construction has recently been proven to achieve vCCA security in [26].

Although a number of precautions need to be taken, the “two-ciphertext” heuristic has also been consistently applied to LWE-based schemes in several works since [12], e.g. [17,41,52,26].

B Proof of Prop. 11 (vCCA security of $\mathcal{P}^{(2)}$)

The vCCA security of $\mathcal{P}^{(2)}$, under the assumption that $\mathcal{P}$ is CPA secure and that $\mathcal{P}^{(2)}$ has the LOH property defined in the previous section, has recently been established in [26, Prop. 10] as a corollary of a more general theorem [26, Prop. 7] stating that any public key correct CPA secure linearly homomorphic scheme that satisfies the LOH assumption is also vCCA secure.

For completeness-sake, we give below an alternative direct proof of this result.

Proof. The proof then starts by one step of game hopping.

First game hop. Let G_0 be the vCCA game against $\mathcal{P}^{(2)}$ and G_1 be the same game as G_0 where we modify the challenger such that, when handling a decryption request on ciphertext c , the new challenger invokes the LOH extractor to verify that,

$$\text{Extract}(c, \text{aux}) \neq \emptyset,$$

rather than checking `ImVer`(c) = `True`. Clearly, the two games cannot be distinguished, unless the adversary is able to create a ciphertext such that `ImVer`(c) = `True` and which is not an affine combination of the well-formed ciphertexts that it generated, in violation with the assumption that $\mathcal{P}^{(2)}$ has the LOH property.

Final reduction. To finalize the proof, we then show that, from an adversary $\mathcal{A}$ against G_0 (or equivalently G_1), we can build an adversary $\mathcal{B}$ against the CPA security of $\mathcal{P}^{(2)}$ (which easily follows from that of $\mathcal{P}^{(26)}$) which uses $\mathcal{A}$ as

²⁵ As already emphasized, this is a slight abuse of terminology.

²⁶ See also [26, Prop. 3].

a subroutine and achieves the same advantage. For the reduction to work, we assume that $\mathcal{A}$ and $\mathcal{B}$ agree on a consistent numbering of the ciphertexts (remark then that ct^Δ is the first generated well-formed ciphertext and is assumed to have number 0). The reduction then proceeds as follows,

- When receiving the *single* challenge request over messages $m_0 \neq m_1 \in \mathbb{Z}_n$ from $\mathcal{A}$, it transfers it as is to the CPA challenger to get ciphertext $\text{ct}^* = \mathcal{P}^{(2)}.\text{Enc}(m_\gamma)$ (for unknown bit γ), which it sends back to $\mathcal{A}$.
- When $\mathcal{A}$ issues a decryption request over ciphertext $\text{ct} \in \mathbb{Z}_{n^2}^2$, $\mathcal{B}$ runs the LOH extractor to get,

$$(\pi, \beta) = \text{Extract}(\text{ct}, \text{aux}).$$

Recall that since the only well-formed ciphertexts that $\mathcal{A}$ can generate from scratch are encryptions of 0, $\beta = 0$. Then, when $\pi = \emptyset$, $\mathcal{B}$ returns $\perp$ to $\mathcal{A}$. When it is not the case, then $\mathcal{B}$ returns $\perp$ whenever $\pi_1 \neq 0$, or π_0 when this is not the case ($\pi_1 = 0$).

Further remark that the LOH extractor works only on ciphertexts generated by the adversary (aux is essentially the trace of execution of $\mathcal{A}$) and *not* over those generated by the challenger. Thus, following Def. 15, we have $m = 2$ and the set $(c_0, \dots, c_{m-1})$ (in that definition) is reduced to $(\text{ct}^\Delta, \text{ct}^*)$ where ct^* is the challenge ciphertext. Hence, for a ciphertext ct independent of ct^* (i.e. such that $\pi_1 = 0$), $\mathcal{P}^{(2)}.\text{Dec}(\text{ct}) = \pi_0$. This follows from the perfect correctness of the Paillier scheme and the fact that, by definition of $\mathcal{P}^{(2)}$, any well-formed ciphertext $\text{ct} = (c_0, c_1)$ which encrypts a linear combination $\sum_i \alpha_i m_i$ is such that there exists r and r' such that,

$$c_0 = (\text{ct}^\Delta.c_0)^{\sum_i \alpha_i m_i} r^n \text{ and } c_1 = (\text{ct}^\Delta.c_1)^{\sum_i \alpha_i m_i} r'^n,$$

yielding $\pi_0 = \sum_i \alpha_i m_i$.

Lastly, since ct^* -dependent ciphertexts are such that $\pi_1 \neq 0$, the above reduction duly replies $\perp$ for all decryption requests on ciphertexts which are byproducts of the challenge ciphertext, consistently with the vCCA game decryption oracle specification. $\square$

C CCA1 security of PEG'_z

In this section, we consider scheme PEG'_z in which the encryption function of PEG_z (which takes inputs in $\llbracket 0, B \rrbracket$) is replaced by that of PEG (that takes inputs in $\mathbb{Z}_n$). Recall that from Eq. (8), the former restriction is artificial since it is easy to generate encryptions of any value in $\mathbb{Z}_n$ under PEG_z by means of its encryption function and its legit homomorphic operators. We now prove that the CCA1 security of PEG'_z follows from that of PEG_z .

Proposition 16. *If there exists an CCA1 adversary $\mathcal{A}$ against PEG'_z , then there exists a CCA1 adversary $\mathcal{B}$ against PEG_z that uses $\mathcal{A}$ as a subroutine and achieves the same advantage.*

Proof. Since PEG'_z and PEG_z have the same decryption function, the reduction transfers all decryption requests from $\mathcal{A}$ as is to its CCA1 challenger against PEG_z . So we only have to care about how the reduction handles the challenge request when either or both m_0 and m_1 fall out of $\llbracket 0, B \rrbracket$, as, otherwise, the reduction just transfers it as is to its challenger. To process a challenge request from $\mathcal{A}$ over arbitrary $m_0 \neq m_1$ from $\mathbb{Z}_n$ (assuming wlog $m_0 < m_1$) rather than $\llbracket 0, B \rrbracket$, the reduction first computes $\alpha = \lceil B(m_1 - m_0)^{-1} \rceil_n$ ($m_1 - m_0$ is invertible modulo n with overwhelming probability and so is α). It then sends a challenge request with messages $m'_0 = 0$ and $m'_1 = B$ to get an encryption c^* of $m'_\beta = \beta B$ (under PEG_z) for unknown bit β . Finally, $\mathcal{B}$ returns,

$$\text{PEG.AddPlain}_{m_0}(\text{PEG.MulByPlain}_{\lceil \alpha^{-1} \rceil_n}(c^*)),$$

which is thus an encryption of m_β (since $\alpha^{-1}m'_0 + m_0 = m_0$ and $\alpha^{-1}m'_1 + m_0 = B^{-1}(m_1 - m_0)B + m_0 = m_1$), to $\mathcal{A}$. Remark that we used PEG 's homomorphic operators which take their plaintext input in $\mathbb{Z}_n$ for simplicity sake, as the setup places no constraints on the postprocessing that the reduction may do on the challenge ciphertext. $\square$

We conclude this section by an informal discussion on the conditions under which the heuristic that we considered at the end of Sect. 5.2, replacing the $\text{Mul}^\mathcal{O}$ oracle in Eq. (11) (using the checked PEG_z decryption function) by the $\text{Mul}_H^\mathcal{O}$ oracle of Eq. (12) (using the unchecked PEG decryption function instead), may be sound. Indeed, we think that $\text{PEG}'_z/\text{Mul}_H^\mathcal{O}$ may be proven CCA1^M secure under the *additional* assumption that PEG'_z is PA1 [10], with PA1 plaintext extractor (which returns values in $\mathbb{Z}_n$) being used in lieu of PEG.Dec to implement $\text{Mul}_H^\mathcal{O}$ in a reduction from a CCA1^M adversary against $\text{PEG}'_z/\text{Mul}_H^\mathcal{O}$ towards a CCA1 challenger against PEG'_z .

Having said this, PA1 is generally assumed in order to prove the CCA1 security of some scheme following the implication that CPA security along with PA1 implies CCA1 security [10]. However, CCA1 does not imply PA1. Hence, although assuming that a CCA1 secure scheme may be PA1 may be less of a leap of faith than assuming that a CPA secure one achieves the same, assuming that PEG'_z is PA1 would be an additional non-falsifiable assumption (on top of DCR).

References

1. Akavia, A., Gentry, C., Halevi, S., Vald, M.: Achievable CCA2 relaxation for homomorphic encryption. In: TCC. pp. 70 – 99 (2022)
2. Akavia, A., Gentry, C., Halevi, S., Vald, M.: Achievable CCA2 relaxation for homomorphic encryption. *Journal of Cryptology* **38** (2025)
3. Alexandru, A., Badawi, A.A., Micciancio, D., Polyakov, Y.: Application-aware approximate homomorphic encryption: Configuring FHE for practical use. *IACR Communications in Cryptology* **2**(4) (2025)

4. Armknecht, F., Katzenbeisser, S., Peter, A.: Group homomorphic encryption: characterizations, impossibility results, and applications. *Designs, Codes and Cryptography* **67**, 209–232 (2013)
5. Barak, B., Pass, R.: On the possibility of one-message weak zero-knowledge. In: *TCC*. pp. 121–132 (2004)
6. Belaïd, S., Bon, N., Boudguiga, A., Sirdey, R., Trama, D., Ye, N.: Further improvements in AES execution over TFHE. *IACR Communications in Cryptology* **2**(1) (2025)
7. Bellare, M., Desai, A., Jokipii, E., Rogaway, P.: A concrete security treatment of symmetric encryption. In: *IEEE SFCS*. pp. 394–403 (1997)
8. Bellare, M., Desai, A., Pointcheval, D., Rogaway, P.: Relations among notions of security for public-key encryption schemes. In: *CRYPTO*. pp. 26–45 (1998)
9. Bellare, M., Palacio, A.: The knowledge-of-exponent assumptions and 3-round zero-knowledge protocols. In: *CRYPTO*. p. 273–289 (2004)
10. Bellare, M., Palacio, A.: Towards plaintext-aware public-key encryption without random oracles. In: *ASIACRYPT*. pp. 48–62 (2004)
11. Bitansky, N., Chiesa, A., Ishai, Y., Ostrovsky, R., Paneth, O.: Succinct non-interactive arguments via linear interactive proofs. *Tech. Rep. 718, IACR ePrint* (2012)
12. Bitansky, N., Chiesa, A., Ishai, Y., Ostrovsky, R., Paneth, O.: Succinct non-interactive arguments via linear interactive proofs. In: *TCC*. pp. 315–333 (2013)
13. Bitansky, N., Chiesa, A., Ishai, Y., Ostrovsky, R., Paneth, O.: Succinct non-interactive arguments via linear interactive proofs. *Journal of Cryptology* **35**, 15–87 (2022)
14. Bitansky, N., Harsha, P., Ishai, Y., Rothblum, R.D., Wu, D.J.: Dot-product proofs and their applications. In: *IEEE FOCS*. pp. 806–825 (2024)
15. Bondarchuk, A., Chakraborty, O., Couteau, G., Sirdey, R.: Downlink (T)FHE ciphertexts compression. In: *SAC*. pp. 647–678 (2025)
16. Boneh, D., Goh, E.J., Nissim, K.: Evaluating 2-DNF formulas on ciphertexts. In: *TCC*. p. 325–341 (2005)
17. Boneh, D., Ishai, Y., Sahai, A., Wu, D.J.: Lattice-based SNARGs and their application to more efficient obfuscation. In: *EUROCRYPT*. p. 247–277 (2017)
18. Boneh, D., Segev, G., Waters, B.: Targeted malleability: homomorphic encryption for restricted computations. In: *ITCS*. pp. 350–366 (2012)
19. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: *CRYPTO*. pp. 868–886 (2012)
20. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. *TOCT* pp. 1–36 (2014)
21. Bresson, E., Catalano, D., Pointcheval, D.: A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications. In: *ASIACRYPT*. pp. 37–54 (2003)
22. Brzuska, C., Canard, S., Fontaine, C., Phan, D.H., Pointcheval, D., Renard, M., Sirdey, R.: Relations among new CCA security notions for approximate FHE. *IACR Communications in Cryptology* **2**(1) (2025)
23. Camenisch, J., Shoup, V.: Practical verifiable encryption and decryption of discrete logarithms. In: *CRYPTO*. pp. 126–144 (2003)
24. Canetti, R., Raghuraman, S., Richelson, S., Vaikuntanathan, V.: Chosen-ciphertext secure fully homomorphic encryption. In: *PKC*. pp. 213–240 (2017)
25. Catalano, D., Fiore, D.: Using linearly-homomorphic encryption to evaluate degree-2 functions on encrypted data. In: *ACM SIGSAC*. pp. 1518–1529 (2015)

26. Checri, M., Clet, P.E., Renard, M., Sirdey, R.: Achieving “beyond CCA1” security for linearly homomorphic encryption, without SNARKs? Tech. Rep. ePrint 2025/894, IACR (2025)
27. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the practical CPAD security of “exact” and threshold FHE schemes. In: CRYPTO. pp. 3–33 (2024)
28. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the IND-CPAD security of exact FHE schemes. In: CCS. pp. 2505 – 2519 (2024)
29. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: ASIACRYPT. pp. 409–437 (2017)
30. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: bootstrapping in less than 0.1 seconds. In: ASIACRYPT (2016)
31. Coppolino, L., D’Antonio, S., Formicola, V., Mazzeo, G., Romano, L.: VISE: Combining Intel SGX and homomorphic encryption for cloud industrial control systems. *IEEE Transactions on Computers* **70**(5), 711–724 (2021)
32. Cramer, R., Shoup, V.: Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM J. Comput.* **33**(1), 167–226 (2003)
33. Damgård, I.: Towards practical public key systems secure against chosen ciphertext attacks. In: CRYPTO. p. 445–456 (1992)
34. D’Antonio, S., Lazarou, G., Mazzeo, G., Stan, O., Zuber, M., Tsavdaridis, I.: The alliance of HE and TEE to enhance their performance and security. In: *IEEE CSR*. pp. 641–647 (2023)
35. Dodis, Y., Halevi, S., Wichs, D.: Security with functional re-encryption from CPA. In: TCC. pp. 279 – 305 (2023)
36. Dwork, C., Naor, M., Reingold, O.: Immunizing encryption schemes from decryption errors. In: EUROCRYPT. pp. 342–360 (2004)
37. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Tech. Rep. 2012/144, IACR ePrint (2012)
38. Fang, Y., Yang, X., Zhu, H., Xu, W., Zheng, Y., Liu, X., Zhang, D.: Enhancing Paillier to fully homomorphic encryption with semi-honest TEE. *Peer-to-Peer Networking and Applications* **17**, 3476–3488 (2024)
39. Fuchsbaauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: CRYPTO. pp. 33–62 (2018)
40. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct NIZKs without PCPs. In: EUROCRYPT. pp. 625–645 (2013)
41. Gennaro, R., Minelli, M., Nitulescu, A., Orrù, M.: Lattice-based zk-SNARKs from square span programs. In: CCS. pp. 556–573 (2018)
42. Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In: STOC. pp. 99–108 (2011)
43. Hada, S., Tanaka, T.: On the existence of 3-round zero-knowledge protocols. In: CRYPTO. pp. 408–423 (1998)
44. Hofheinz, D.: All-but-many lossy trapdoor functions. In: EUROCRYPT. pp. 209–227 (2012)
45. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: EUROCRYPT. pp. 648–677 (2021)
46. Li, B., Micciancio, D., Schultz, M., Sorrell, J.: Securing approximate homomorphic encryption using differential privacy. In: CRYPTO. pp. 560–589 (2022)
47. Libert, B.: Leveraging small message spaces for CCA1 security in additively homomorphic and BGN-type encryption. In: EUROCRYPT. p. 34–63 (2025)
48. Lipmaa, H.: On the CCA1-security of ElGamal and Damgård ElGamal. In: *In-scrypt*. pp. 18–35 (2010)

49. Loftus, J., May, A., Smart, N.P., Vercauteren, F.: On CCA-secure somewhat homomorphic encryption. In: SAC. pp. 55–72 (2011)
50. Manulis, M., Nguyen, J.: Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In: EUROCRYPT. pp. 63–93 (2024)
51. Naor, M.: On cryptographic assumptions and challenges. In: CRYPTO. pp. 96–109 (2003)
52. Nitulescu, A.: Lattice-based zero-knowledge SNARGs for arithmetic circuits. In: LATINCRYPT. pp. 217–236 (2019)
53. Orlandi, C., Scholl, P., Yakubov, S.: The rise of Paillier: Homomorphic secret sharing and public-key silent OT. In: EUROCRYPT. pp. 678–708 (2021)
54. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: EUROCRYPT. pp. 223–238 (1999)
55. Pointcheval, D.: Personal communication (2025)
56. Roy, L., Singh, J.: Large message homomorphic secret sharing from DCR and applications. In: CRYPTO. pp. 687–717 (2021)
57. Schäge, S.: New limits of provable security and applications to ElGamal encryption. In: EUROCRYPT. p. 255–285 (2024)
58. Schäge, S., Vorstermans, M.: New limits for homomorphic encryption. In: ASIACRYPT. pp. 482–515 (2025)
59. Shinozaki, T., Tanaka, K., Tezuka, M., Yoshida, Y.: On the relationship between FuncCPA and FuncCPA+. Tech. Rep. 2024/1166, IACR ePrint (2024)
60. Wang, W., Jiang, Y., Shen, Q., Huang, W., H., C., Wang, S., Wang, X., H., T., Chen, K., Lauter, K., Lin, D.: Toward scalable fully homomorphic encryption through light trusted computing assistance. Tech. Rep. 1905.07766, arXiv (2019)
61. Yang, R.: Personal communication (2025)
62. Yang, R., Yu, Z., Susilo, W.: Fully homomorphic encryption with chosen-ciphertext security from LWE. In: CRYPTO. p. 371–405 (2025)

D Full definition of $\text{CF}^{(2)}$

Following Sect. 6.3, we give the full definition of scheme $\text{CF}^{(2)}$, which is obtained by instantiating the Catalano-Fiore blueprint of Sect. 4.2 but using this time $\mathcal{P}^{(2)}$ as the base scheme. *Considering the alternative encryption function in Eq. (7)*, the scheme is then defined as follows:

- $\text{CF}^{(2)}.\text{KeyGen}$: run $\mathcal{P}^{(2)}.\text{KeyGen}$.
- $\text{CF}^{(2)}.\text{Enc}$: given $m \in \mathbb{Z}_n$ pick b uniformly in $\mathbb{Z}_n$ and return,

$$\text{ct} = (c_0, c_1) = (b, \mathcal{P}^{(2)}.\text{Enc}(m - b)). \quad (21)$$

- $\text{CF}^{(2)}.\text{Add}$: given two (first form) ciphertexts $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$ and $(c'_0, c'_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$ return (first form) ciphertext,

$$(c''_0, c''_1) = ([c_0 + c'_0]_n, \mathcal{P}^{(2)}.\text{Add}(c_1, c'_1)).$$

- $\text{CF}^{(2)}.\text{Mul}$: given two (first form) ciphertexts $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$ and $(c'_0, c'_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$ return (second form) ciphertext, (c''_0, c''_1, c''_2) , which components are,

$$(\mathcal{P}^{(2)}.\text{Add}(\mathcal{P}^{(2)}.\text{Enc}(c_0 c'_0), \mathcal{P}^{(2)}.\text{MulByPlain}(c_1, c'_0), \mathcal{P}^{(2)}.\text{MulByPlain}(c'_1, c_0)), c_1, c'_1) \in \mathbb{Z}_{n^2}^6.$$

- $\text{CF}^{(2)}.\text{ImVer}_1$ (first form): given $(c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$, return $\mathcal{P}^{(2)}.\text{ImVer}(c_1)$.
- $\text{CF}^{(2)}.\text{ImVer}_2$ (second form): given $(c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, return True if $\mathcal{P}^{(2)}.\text{ImVer}(c_i) = \text{True}$ for all $i \in \{0, 1, 2\}$.
- $\text{CF}^{(2)}.\text{Dec}_1$ (first form): given $\text{ct} = (c_0, c_1) \in \mathbb{Z}_n \times \mathbb{Z}_{n^2}^2$, return $[c_0 + \mathcal{P}^{(2)}.\text{Dec}(c_1)]_n$ if $\text{CF}^{(2)}.\text{ImVer}_1(\text{ct}) = \text{True}$ and $\perp$ otherwise.
- $\text{CF}^{(2)}.\text{Dec}_2$ (second form): given $\text{ct} = (c_0, c_1, c_2) \in \mathbb{Z}_{n^2}^6$, return,

$$[\mathcal{P}^{(2)}.\text{Dec}(c_0) + \mathcal{P}^{(2)}.\text{Dec}(c_1)\mathcal{P}^{(2)}.\text{Dec}(c_2)]_n,$$

if $\text{CF}^{(2)}.\text{ImVer}_2(\text{ct}) = \text{True}$ and $\perp$ otherwise.

E Additional security notions and assumptions

CPA and CCA1 security. For completeness, we quickly recall the standard notions of CPA and CCA1 security. Note that some cryptographers consider that CCA1 security is a historical security notion from the time before the first CCA2 public encryption schemes were proposed. While this is a legitimate point of view, CCA1 may still be considered a meaningful CCA security target for homomorphic schemes in order to circumvent the CCA2 impossibility to which malleable schemes are subject [49, 24].

In the *CPA game*, the adversary only has access to an encryption oracle and a challenge oracle, the latter of which it can invoke only once. Stricto sensu the encryption oracle is necessary only in the private key case as, in the public key case, the adversary can generate ciphertexts on its own. When dealing with homomorphic schemes, the adversary also performs homomorphic evaluations on its own in both the private and public key cases.

In the *CCA1 game*, the adversary can also issue encryption requests and perform a single challenge request. Before this unique challenge request, the adversary is additionally granted access to a decryption oracle which simply returns $\mathcal{E}.\text{Dec}(c)$ for any arbitrary ciphertexts $c \in \mathcal{C}$. Then, after the single challenge request has been replied to, this decryption oracle systematically replies $\perp$. Again, for homomorphic schemes, the CCA1 game has no evaluation oracle as the adversary performs the homomorphic evaluations on its own.

For completeness, we remind below the formal definitions of the CPA and CCA1 experiments (echoing our generic experiment on p. 8).

Definition 17 (CPA experiment).

$$\begin{aligned}
(pk, sk) &\leftarrow \mathcal{E}_H.\text{KeyGen}(1^\lambda), \\
(m_0, m_1, S) &\leftarrow \mathcal{A}_1^{[\mathcal{E}_H.\text{Enc}(\cdot)]}(pk), \\
\gamma &\xleftarrow{\$} \{0, 1\}, \\
c^* &\leftarrow \mathcal{E}_H.\text{Enc}(m_\gamma), \\
b' &\leftarrow \mathcal{A}_2^{[\mathcal{E}_H.\text{Enc}(\cdot)]}(pk, S, c^*), \\
b &= b'.
\end{aligned}$$

Definition 18 (CCA1 experiment).

$$\begin{aligned}
(pk, sk) &\leftarrow \mathcal{E}_H.\text{KeyGen}(1^\lambda), \\
(m_0, m_1, S) &\leftarrow \mathcal{A}_1^{[\mathcal{E}_H.\text{Enc}(\cdot)], \mathcal{E}_H.\text{Dec}(\cdot)}(pk), \\
\gamma &\stackrel{\$}{\leftarrow} \{0, 1\}, \\
c^* &\leftarrow \mathcal{E}_H.\text{Enc}(m_\gamma), \\
b' &\leftarrow \mathcal{A}_2^{[\mathcal{E}_H.\text{Enc}(\cdot)]}(pk, S, c^*), \\
b &= b'.
\end{aligned}$$

CPA^D security Another security notion which is foundational for homomorphic encryption is that of CPA^D security [45]. This notion is a slight generalization of CPA security²⁷, in which the CPA adversary is further granted access to a decryption oracle accepting only well-formed ciphertexts, that is either well-formed fresh ciphertexts obtained by genuinely running the encryption function or ciphertexts derived from fresh well-formed ciphertexts via legit homomorphic evaluations. To do so, the CPA^D game provides encryption and evaluation oracles (even in the public-key setting) which purpose is to fill a game state with well-formed ciphertexts for handling subsequent evaluation or decryption requests on state indices. For *correct* encryption schemes, it is known that CPA^D security collapses onto CPA security [45]. When this assumption does not hold, which is the case for LWE- or RLWE-based schemes, CPA^D security has to be dealt with explicitly [45, 27, 28].

Computational assumptions.

Definition 19 (DCR, [47, 54]). Let $N = pq$, the *Decision Composite Residuosity assumption* states that the distributions $\{[z^N]_{N^2} : z \stackrel{\$}{\leftarrow} \mathbb{Z}_N^*\}$ and $\{z \stackrel{\$}{\leftarrow} \mathbb{Z}_{N^2}^*\}$ are computationally indistinguishable.

Compared to the above, the Composite Non-Invertibility (CNI) assumption is less standard although it is falsifiable and has been around for several years [44, Assumption 4.4]. The name CNI has been coined by Libert [47, Def. 1]. Informally, this assumption states that given a RSA modulus $N = pq$, an adversary cannot create an encryption of a multiple of either p or q under the Paillier scheme.

Definition 20 (CNI, [47, 44]). Let $\mathcal{P} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ denote the Paillier scheme (Sect. F), the *Composite Non-Invertibility assumption* states that for any PPT adversary $\mathcal{A}$ we have,

$$P\left(\mu = \mathcal{P}.\text{Dec}(c) \wedge \begin{array}{l} ((N, y), \omega) := \mathcal{P}.\text{KeyGen}(1^\lambda) \\ c := \mathcal{A}(N) \end{array} \mid 1 < \gcd(\mu, N) < N\right) \leq \text{neg}(\lambda).$$

²⁷ Contrary to CPA, the CPA^D game by default allows for multiple challenge requests. While single- and multiple challenge variants of CPA security are equivalent [8, 7], this is not the case for CPA^D [22].

F The Paillier cryptosystem

This section briefly presents Paillier's original cryptosystem [54], denoted by $\mathcal{P}$ in this paper, which CPA security is grounded in the Composite Residuosity Class Problem hardness assumption. The scheme is partially homomorphic allowing additions, or multiplications by a constant, but does not support multiplications between two ciphertexts. Note that a relatively simple modification of this scheme, described in [25], allows to perform one level of multiplication (see also, Sect. 4.2).

Let n be an RSA modulus. The plaintext space is $\mathbb{Z}_n$ and the ciphertext space is $\mathbb{Z}_{n^2}^\times$. Let $\mathcal{S}_n$ be the set $\mathcal{S}_n = \{u \in \mathbb{Z}_{n^2}^\times \mid u \equiv 1 \pmod{n}\}$, which is a multiplicative subgroup of $\mathbb{Z}_{n^2}^\times$. For all $u \in \mathcal{S}_n$ we define the function $L : \mathcal{S}_n \rightarrow \mathbb{Z}_n$, such that $L(u) = \frac{u-1}{n}$.

- $\mathcal{P}.$ KeyGen: sample a RSA modulus $n = pq$ such that p and q are distinct large prime numbers and such that $\gcd(pq, (p-1)(q-1)) = 1$. Let $\varphi(n) = (p-1)(q-1)$ and $\omega := \omega(n) = \text{lcm}(p-1, q-1)$. Choose uniformly at random an integer $g \in \mathbb{Z}_{n^2}^\times$, such that $\gcd(L(g^\omega \pmod{n^2}), n) = 1$. Set the public key $\text{pk} = (n, g)$ and the secret key $\text{sk} = \omega(n)$.
- $\mathcal{P}.$ Enc: given $m \in \mathbb{Z}_n$ and pk , sample uniformly at random $r \xleftarrow{\$} (\mathbb{Z}_n)^\times$ and return $c = g^m r^n \pmod{n^2}$.
- $\mathcal{P}.$ Dec: given $c \in \mathbb{Z}_{n^2}$ and sk , return $\frac{L(c^{\text{sk}} \pmod{n^2})}{L(g^{\text{sk}} \pmod{n^2})} \pmod{n}$.
- $\mathcal{P}.$ Add: given $c, c' \in \mathbb{Z}_{n^2}$, compute and return $[c \cdot c']_{n^2}$.
- $\mathcal{P}.$ MulByPlain: given $\alpha \in \mathbb{Z}_n$ and $c \in \mathbb{Z}_{n^2}$, compute and return $[c^\alpha]_{n^2}$.

As a notable property with respect to the present work, the Paillier scheme achieves perfect correctness.